

Projeto e Análise de Algoritmos

Menor caminho em Grafos

Thiago Pinheiro de Araújo

EMAp/FGV

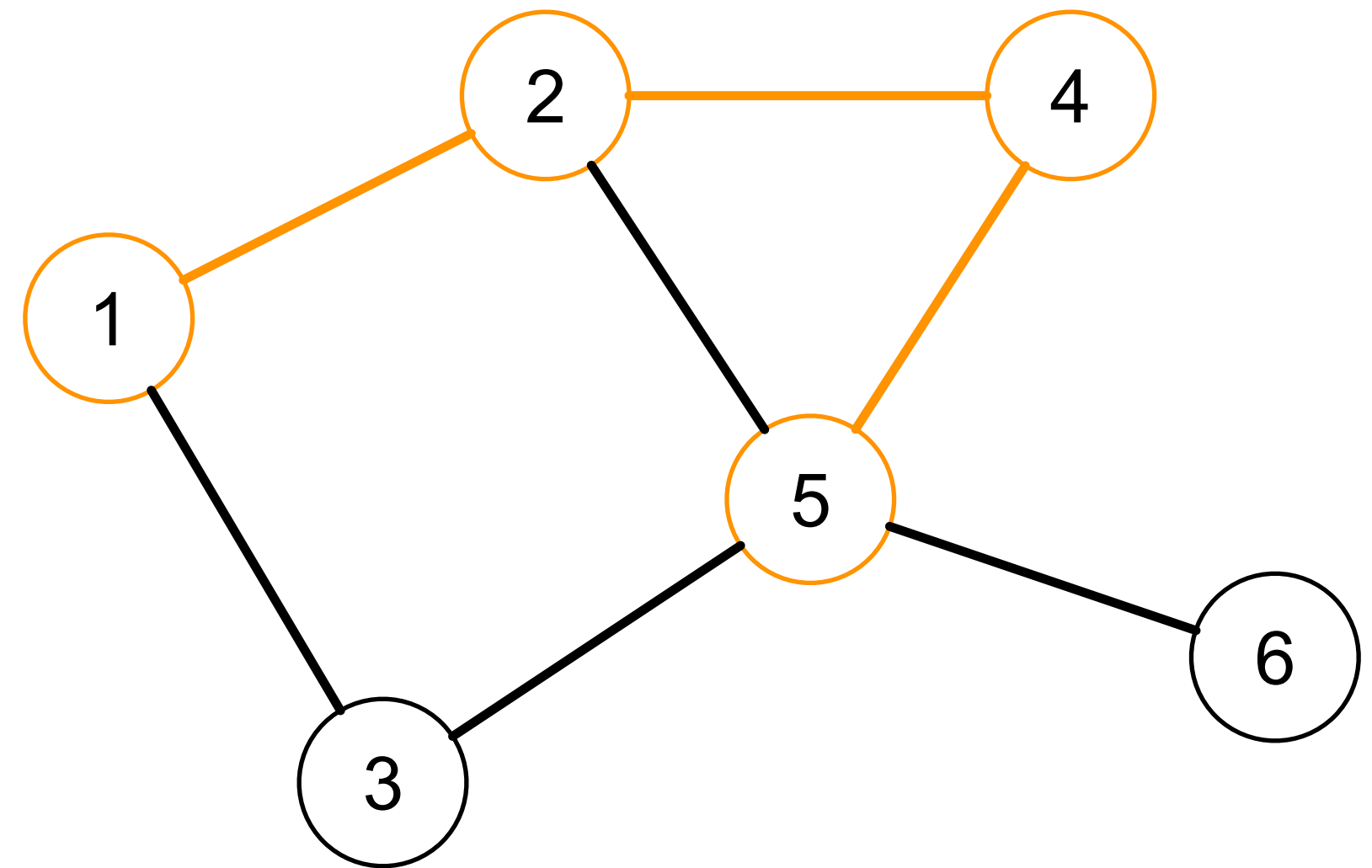
2025.2

Apresentar algoritmos para encontrar **caminhos em grafos**, discutir as suas características, analisar a complexidade e demonstrar sua implementação.

Menor caminho em Grafos

Introdução

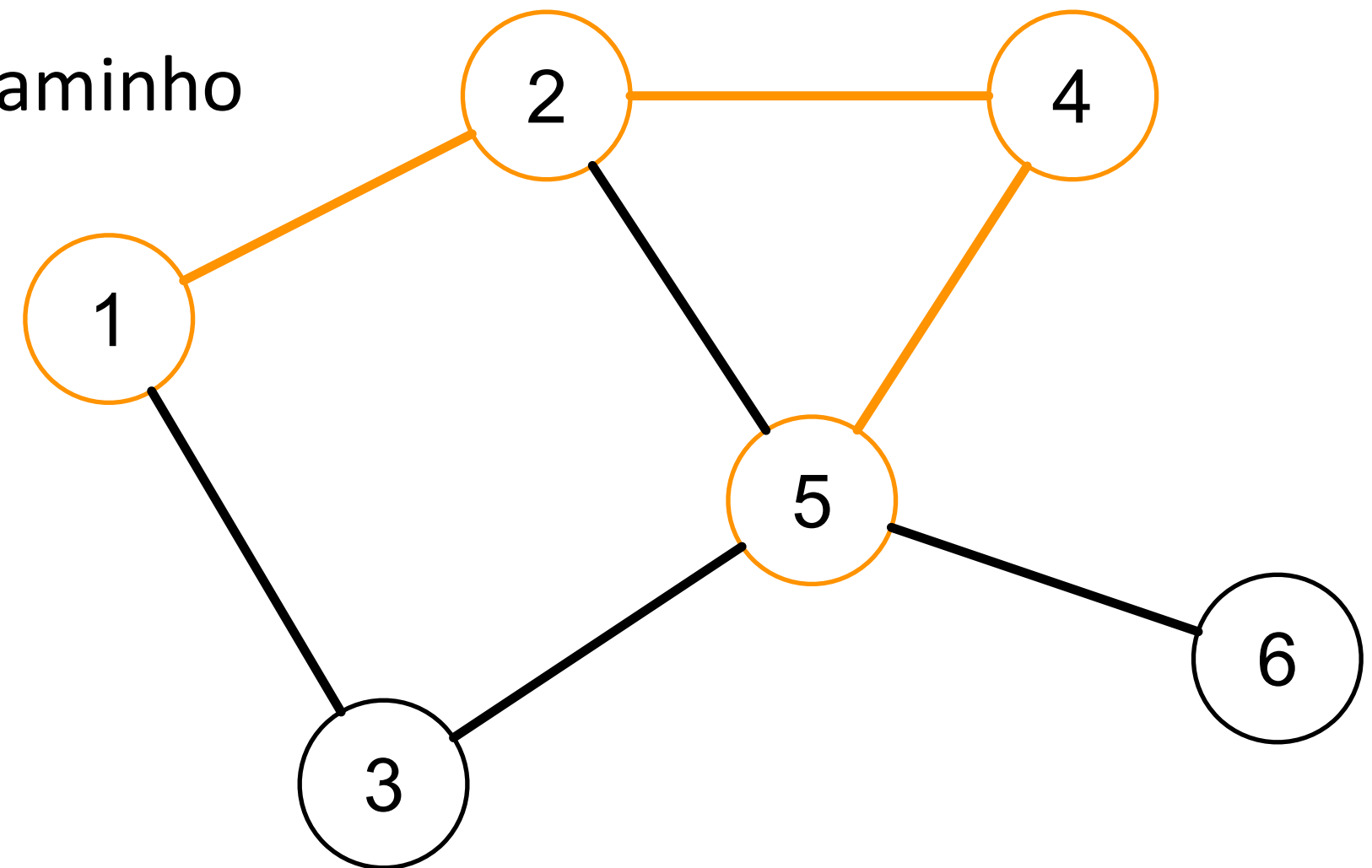
- Um **caminho** P em $G = (V, E)$ consiste em uma sequência com n vértices, finita, tal que v_{i+1} é adjacente de v_i para todo $i < n$.
- O caminho é simple se não possuir vértices repetidos.
- O caminho é fechado (ciclo) se $v_1 = v_n$.
- O comprimento de um caminho é definido pelo número de arestas presentes nele
 - Um caminho que passa por n vértices possui $n - 1$ arestas.



Menor caminho em Grafos

Caminho mais curto

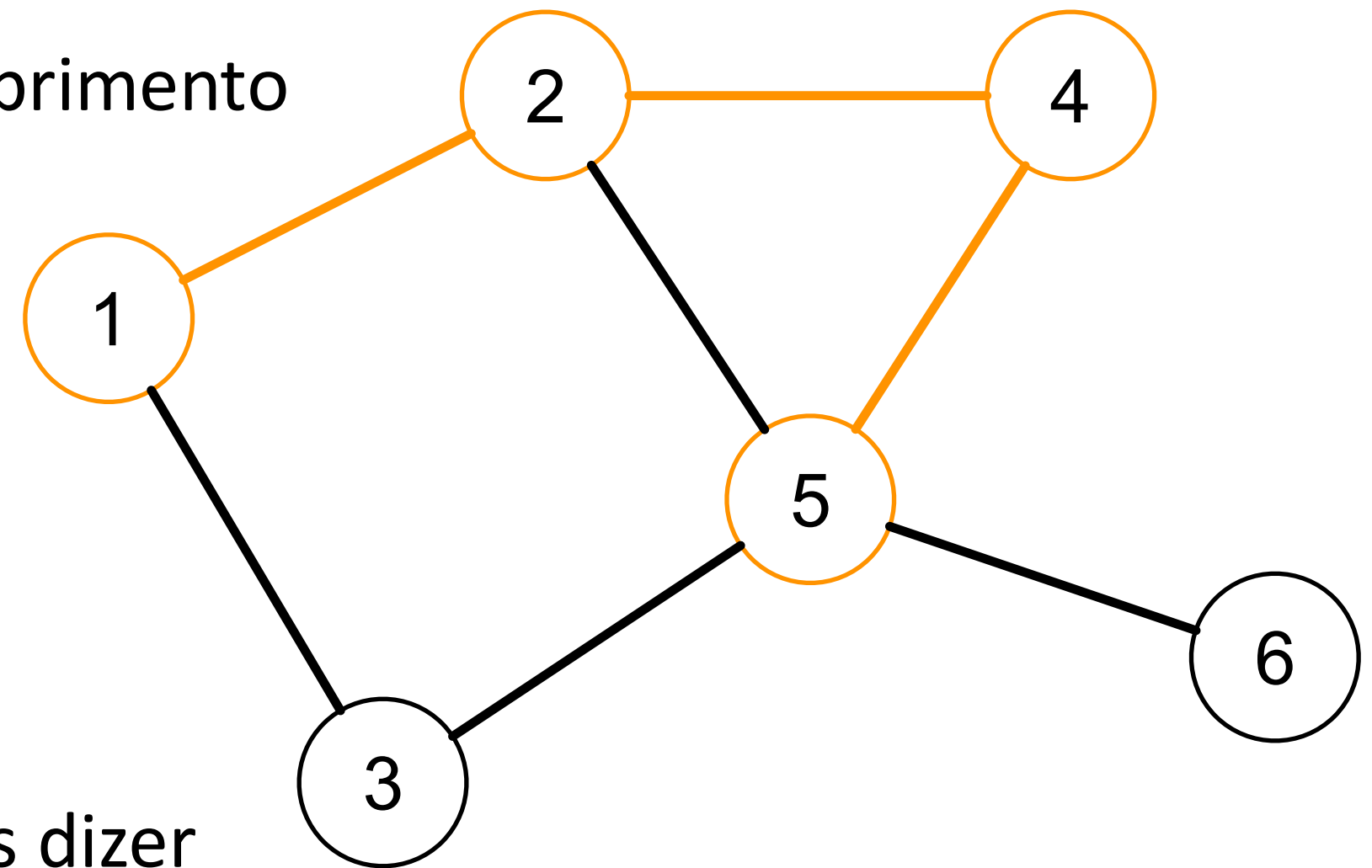
- **Problema:** dados dois vértices v_i e v_j em um grafo $G = (V, E)$ encontre o caminho mínimo P que começa em v_i e termina em v_j .
- O caminho é mínimo caso não exista outro caminho em G entre os vértices v_i e v_j cujo número de arestas seja menor que o de P .
- Observe que pode não existir um caminho entre v_i e v_j .



Menor caminho em Grafos

Caminho mais curto

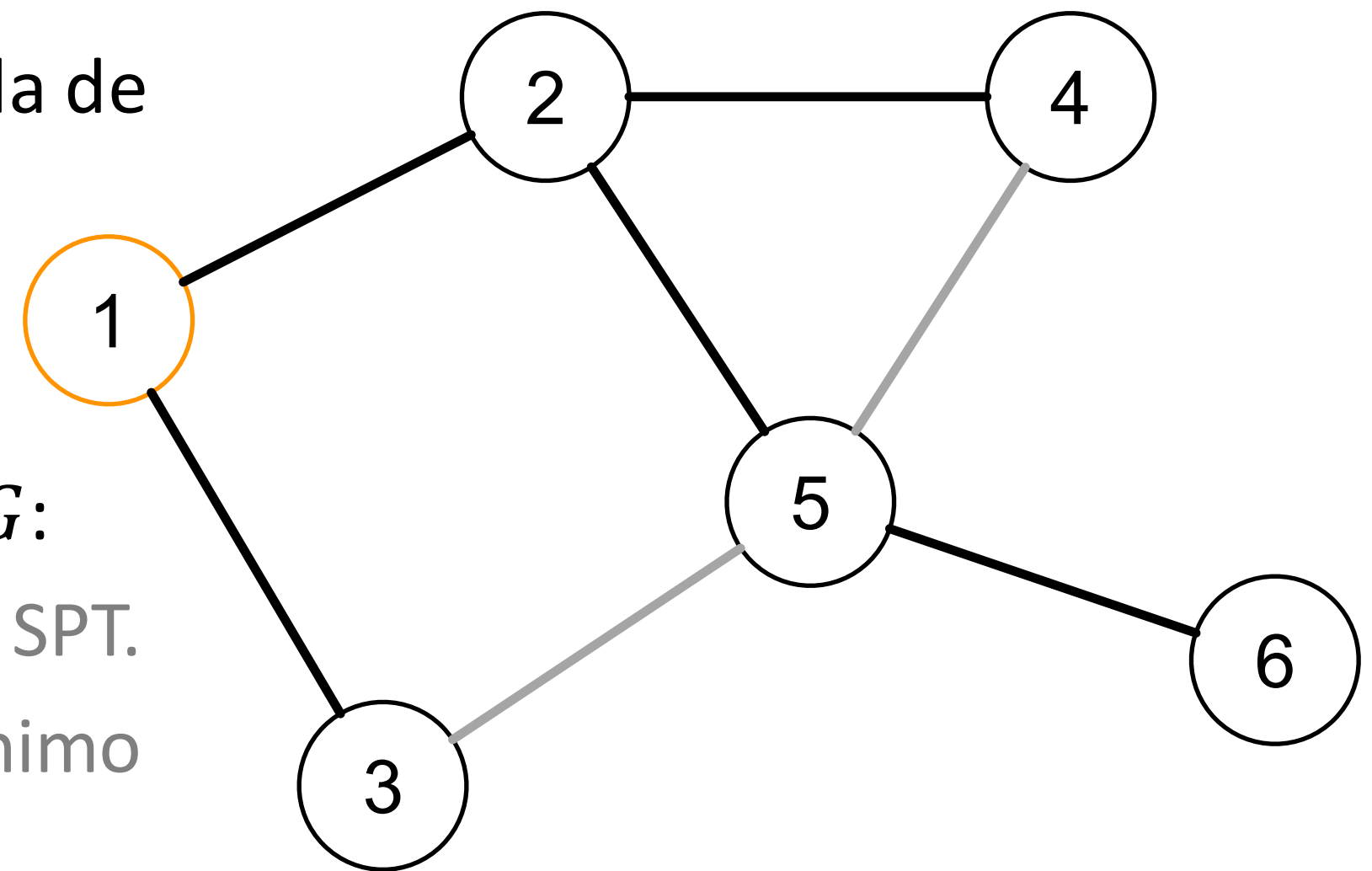
- **Problema:** dados dois vértices v_i e v_j em um grafo $G = (V, E)$ encontre o caminho mais curto P que começa em v_i e termina em v_j .
- A distância entre v_i e v_j é definida pelo comprimento do caminho mais curto.
- A distância entre v_i e v_j pode ser diferente da distância entre v_j e v_i caso o grafo seja orientado.
- Se não existe caminho entre v_i e v_j podemos dizer que a distância é infinita.



Menor caminho em Grafos

Caminho mais curto

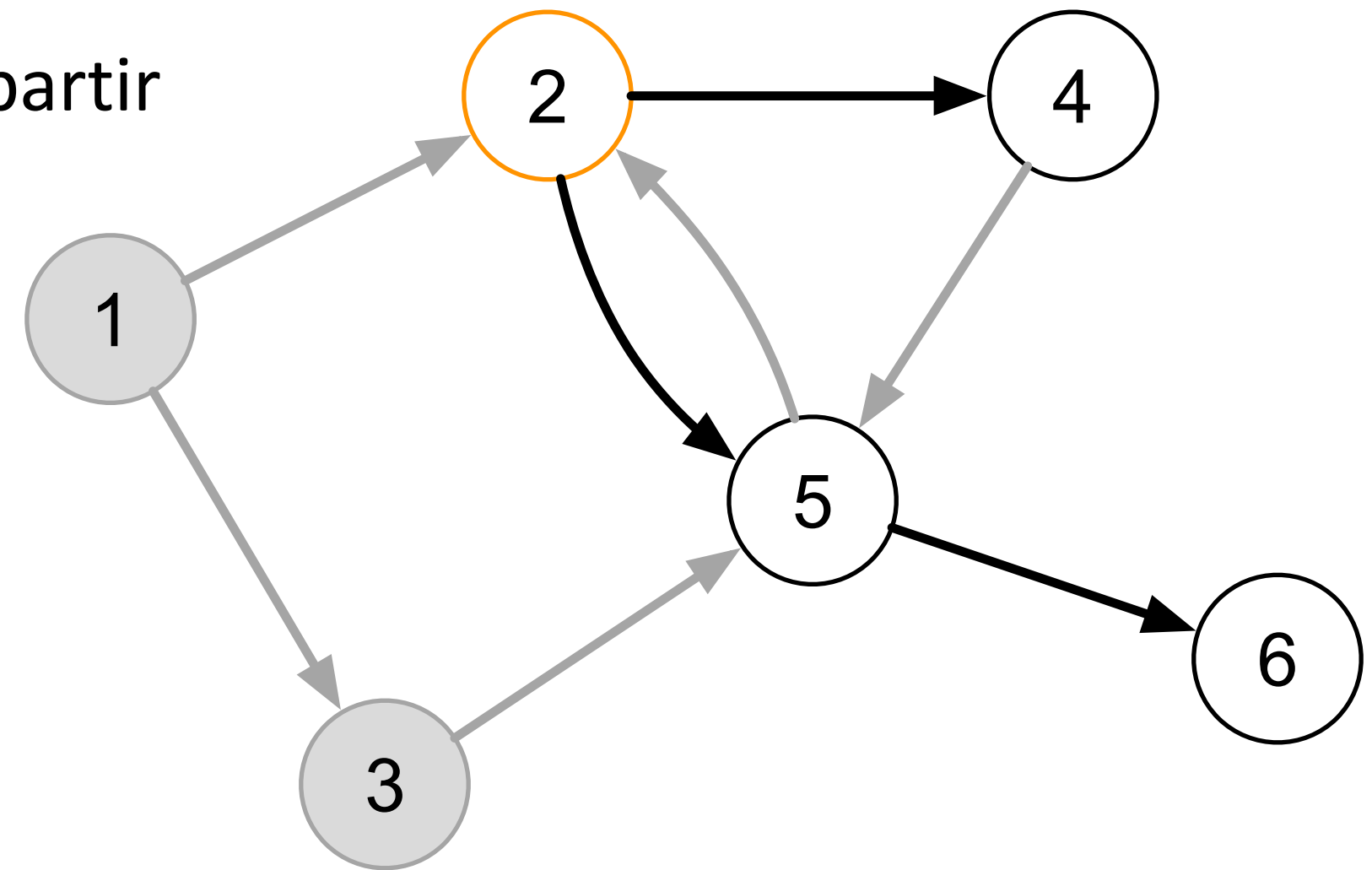
- Para encontrar o **caminho mais curto** entre v_i e v_j é inevitável encontrar todos os caminhos que iniciam em v_i .
- A árvore produzida por essa busca é chamada de Árvore de caminhos mais curtos (SPT – *Shortest Path Tree*).
- A árvore SPT é uma sub-árvore radicada de G :
 - Todos os vértices de G estão presentes na SPT.
 - Todo caminho na SPT a partir da raiz é mínimo no grafo G .



Menor caminho em Grafos

Caminho mais curto

- Um grafo somente possui uma SPT caso todos os vértices sejam acessíveis a partir de v_i .
- Caso não exista uma SPT para um grafo G a partir de uma raiz v_i , existe uma SPT para um sub-grafo induzido de G com os vértices acessíveis a partir de v_i .
- Portanto, para encontrar o menor caminho entre v_i e v_j precisamos encontrar a árvore radicada desse sub-grafo com raiz em v_i .

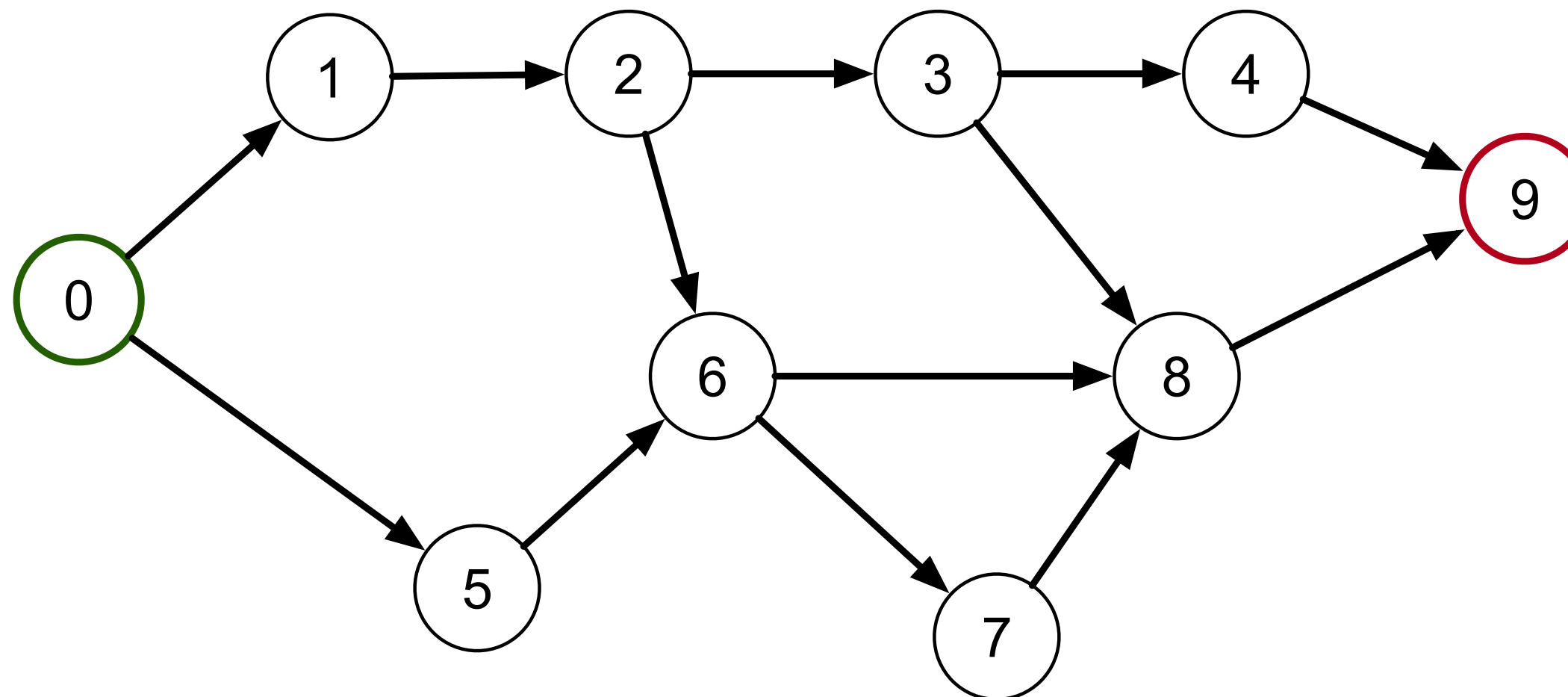


Busca pelo caminho mais curto em um DAG

Menor caminho em Grafos

Caminho mais curto em um DAG

- **Problema:** como criar um algoritmo capaz de gerar a SPT de um DAG iniciando na sua única fonte?
- Considere que você possui uma possível ordem topológica para o DAG.
- Dica: utilize as propriedades do DAG (Grafo Direcionado Acíclico).

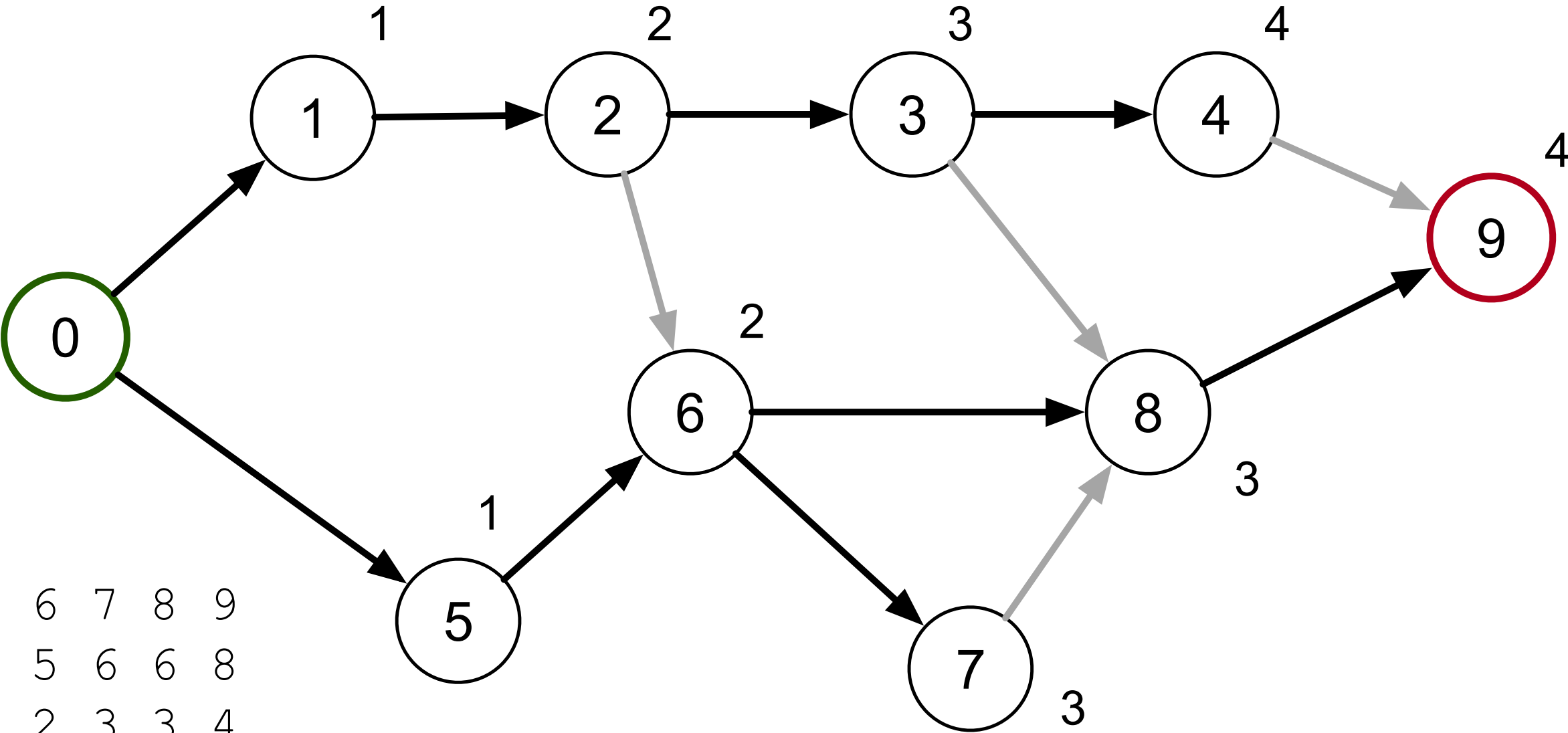


- **Problema:** como criar um algoritmo capaz de gerar a SPT de um DAG iniciando na sua única fonte?
- Solução:
 - Inicialize cada vértice com distância infinita para a raiz ($d[v_i] = \infty$) e pai indefinido ($parent[v_i] = -1$).
 - Defina a raiz com distância zero ($d[v_1] = 0$).
 - Percorra os vértices seguindo a ordem topológica
 - Avalie para cada vértice adjacente se $d[v_i] + 1 \leq d[v_j]$.
 - Se for atualiza $d[v_j]$ no vértice adjacente com a menor distância.
 - E define o novo pai do vértice adjacente ($parent[v_j] = v_i$)

Menor caminho em Grafos

Caminho mais curto em um DAG

- O resultado ao executar sobre o grafo a seguir é:

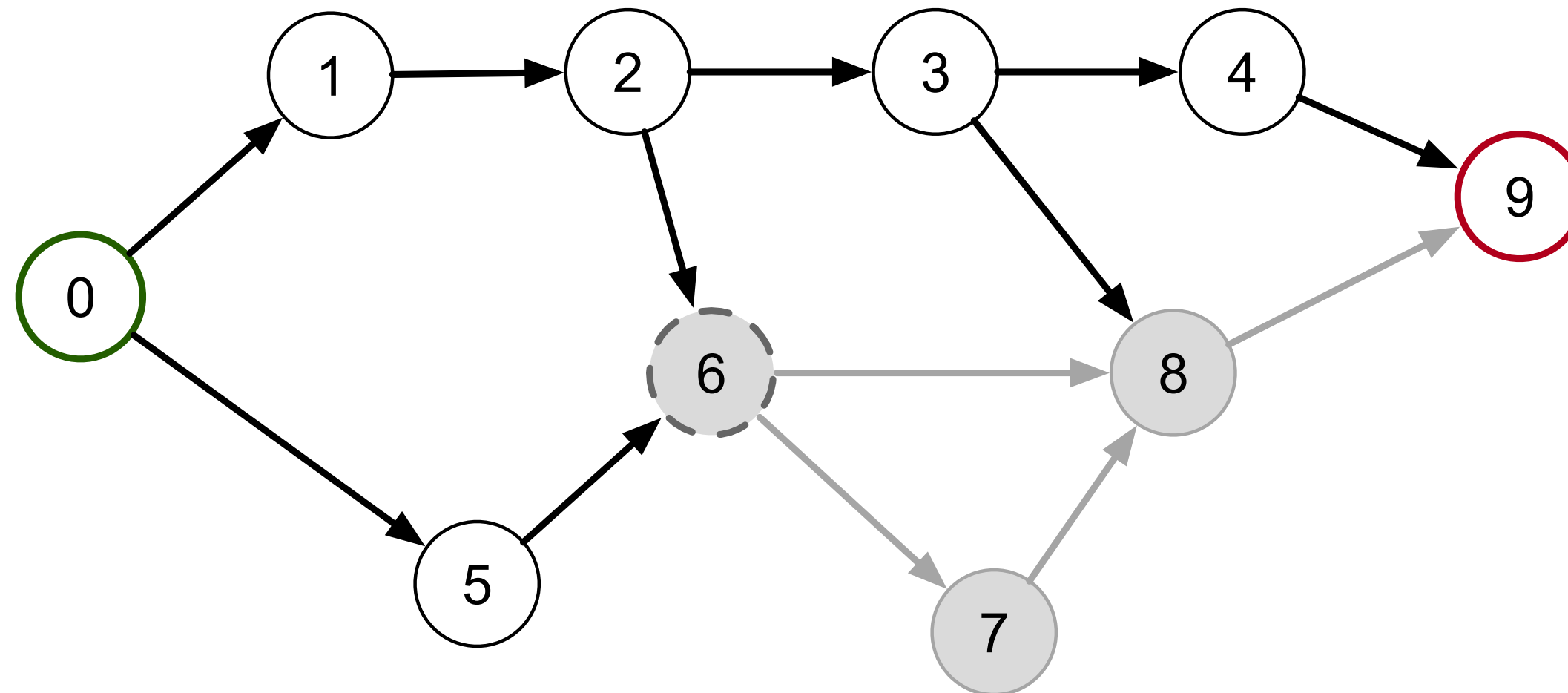


v	0	1	2	3	4	5	6	7	8	9
parent[v]	0	0	1	2	3	0	5	6	6	8
distance[v]	0	1	2	3	4	1	2	3	3	4

Menor caminho em Grafos

Caminho mais curto em um DAG

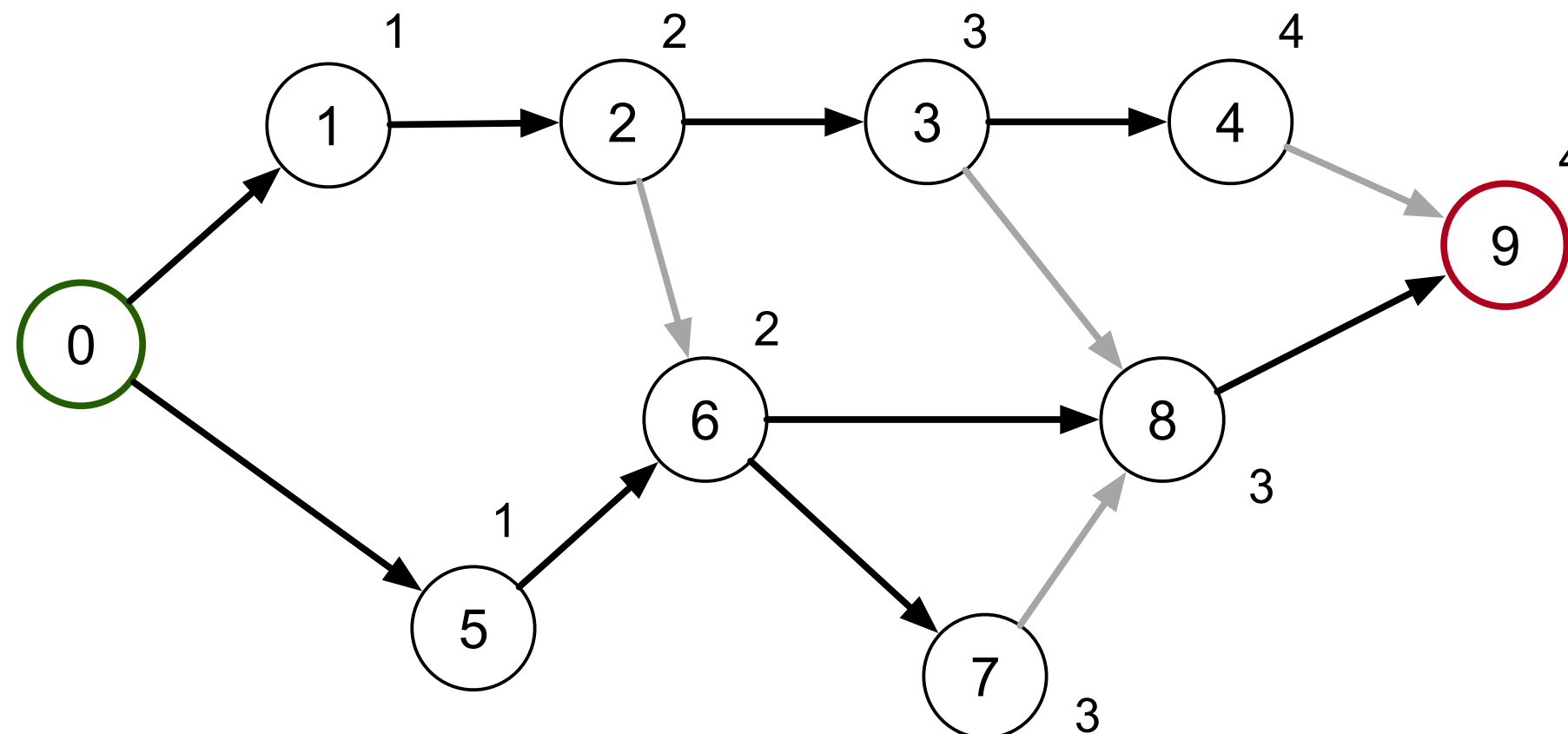
- Podemos avaliar a corretude desse algoritmo através das suas invariantes:
 - O vetor *parent* define uma árvore radicada T com raiz em v_0 .
 - Para todo vértice v_k em G , $d[v_k]$ é a distância de v_0 até v_k em T .
 - Para toda aresta $e_k = (v_i, v_j)$ de G , se v_i foi processado então e_k já foi avaliada.



Menor caminho em Grafos

Caminho mais curto em um DAG

- Ao término da execução:
 - T é uma árvore radcada de um grafo H , induzido de G , contendo os vértices acessíveis a partir de v_0 .
 - Toda aresta de H foi avaliada e T é uma árvore geradora de H .
 - O valor $d[v_k]$ representa a menor distância entre o vértice v_k e a raiz da árvore.



- **Exercício:** proponha uma implementação para o algoritmo apresentado nos slides anteriores (considere a numeração dos vértices topológica).

```
void dagSPT(vertex * parent, int * distance) {
    const int INFINITY = m_numVertices;
    for (vertex v=0; v < m_numVertices; v++) {
        distance[v] = INFINITY;
        parent[v] = -1;
    }
    distance[0] = 0;
    parent[0] = 0;
    for (int v1=0; v1 < m_numVertices; v1++) {
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (distance[v1] + 1 < distance[v2]) {
                distance[v2] = distance[v1] + 1;
                parent[v2] = v1;
            }
            edge = edge->next();
        }
    }
}
```

- Analise da complexidade:
 - Cada vértice é processado uma única vez.
 - Em cada vértice v_i são verificadas $g_s(v_i)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $\Theta(V + E)$ ao utilizar a estrutura de dados lista de adjacências
 - Se a estrutura de dados for matriz de adjacências a complexidade vai ser $\Theta(V^2)$.
- Se o grafo for denso, em ambas as estruturas de dados a complexidade será $\Theta(V^2)$.

**Busca pelo caminho
mais curto**

- A comparação $d[v_i] + 1 \leq d[v_j]$ e a eventual atualização no vetor de distâncias é conhecida como operação de relaxamento.
- Dizemos que uma aresta está relaxada se $d[v_j] - d[v_i] \leq 1$.
- E tensa se $d[v_j] - d[v_i] > 1$.
- Chamamos de potencial relaxado uma numeração para os vértices que torne todas as aresta do grafo relaxadas
 - O vetor de distâncias resultante do algoritmo anterior é um potencial relaxado.

Caminho mais curto

- **Voltando ao problema inicial:** desejamos encontrar o caminho mais curto entre dois vértices em qualquer grafo.
- Como produzir uma solução para grafos não-dirigidos e/ou que possuem ciclos?
- Solução: podemos adaptar o algoritmo de busca em largura (BFS) de forma que a numeração dos vértices represente a distância para a raiz
 - A fila de vértices passaria a representar um papel semelhante à ordenação topológica, utilizada para guiar a varredura no algoritmo anterior.

Caminho mais curto

- Ideia geral: modificar o ciclo do BFS para:
 - Remover o vértice v_i da fila.
 - Visitar seus vértices adjacentes
 - Se $distance[v_j]$ não estiver definida
 - $distance[v_j] = distance[v_i] + 1$
 - $parent[v_j] = v_i$
 - Inserir v_j na fila
- Diferente do algoritmo anterior, o valor $distance[v]$ é alterado uma única vez.

Menor caminho em Grafos

Caminho mais curto

- **Exercício:** implemente a adaptação do algoritmo BFS para encontrar o caminho mais curto entre um vértice e todos os acessíveis por ele.
- A complexidade é:
 - $O(V + E)$.

```
void spt(vertex v0, vertex * parent, int * distance) {
    const int INFINITY = m_numVertices;
    for (vertex v=0; v < m_numVertices; v++) {
        distance[v] = INFINITY;
        parent[v] = -1;
    }
    distance[v0] = 0;
    parent[v0] = v0;
    queue<int> queue;
    queue.push(v0);
    while (!queue.empty()) {
        int v1 = queue.front();
        queue.pop();
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (distance[v2] == INFINITY) {
                distance[v2] = distance[v1] + 1;
                parent[v2] = v1;
                queue.push(v2);
            }
            edge = edge->next();
        }
    }
}
```

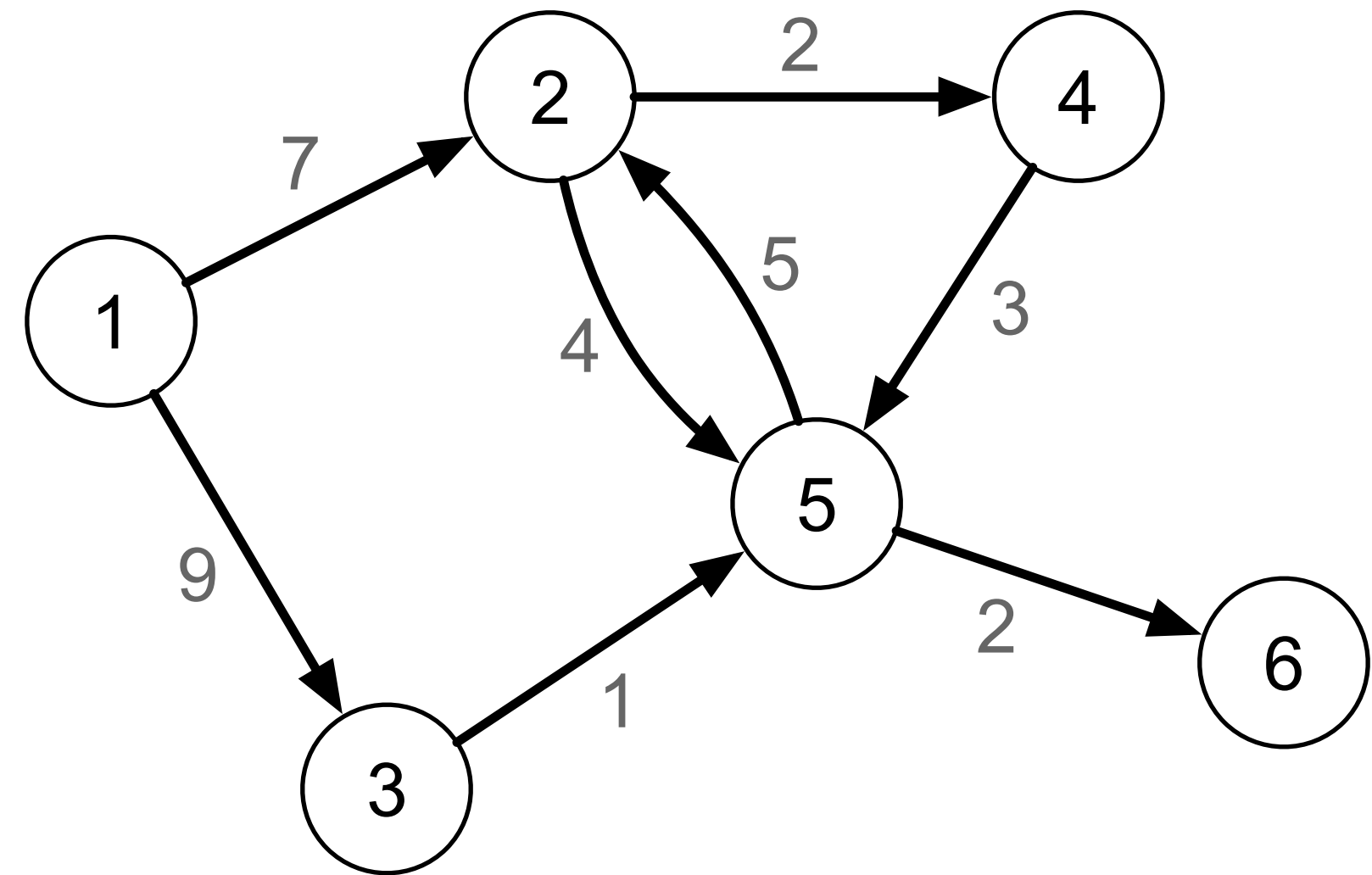
- Podemos avaliar a corretude desse algoritmo através das suas invariantes:
 1. Toda aresta (v_i, v_j) de T (árvore definida por parent) está relaxada com relação à *distance*.
 2. Para cada aresta (v_i, v_j) , se v_i está em T e v_j está fora de T , então v_i está na fila.
- Ao término da execução:
 - A fila está vazia.
 - A partir da invariante (2) toda aresta com v_i em T também possui v_j em T .
 - O vetor *distance* é um potencial relaxado.
 - Portanto, T é uma SPT e *distance* fornece o comprimento do caminho entre a raiz e os demais vértices acessíveis a partir dela.

**Busca pelo caminho
mais barato**

Introdução à Grafos

Caminho mais barato

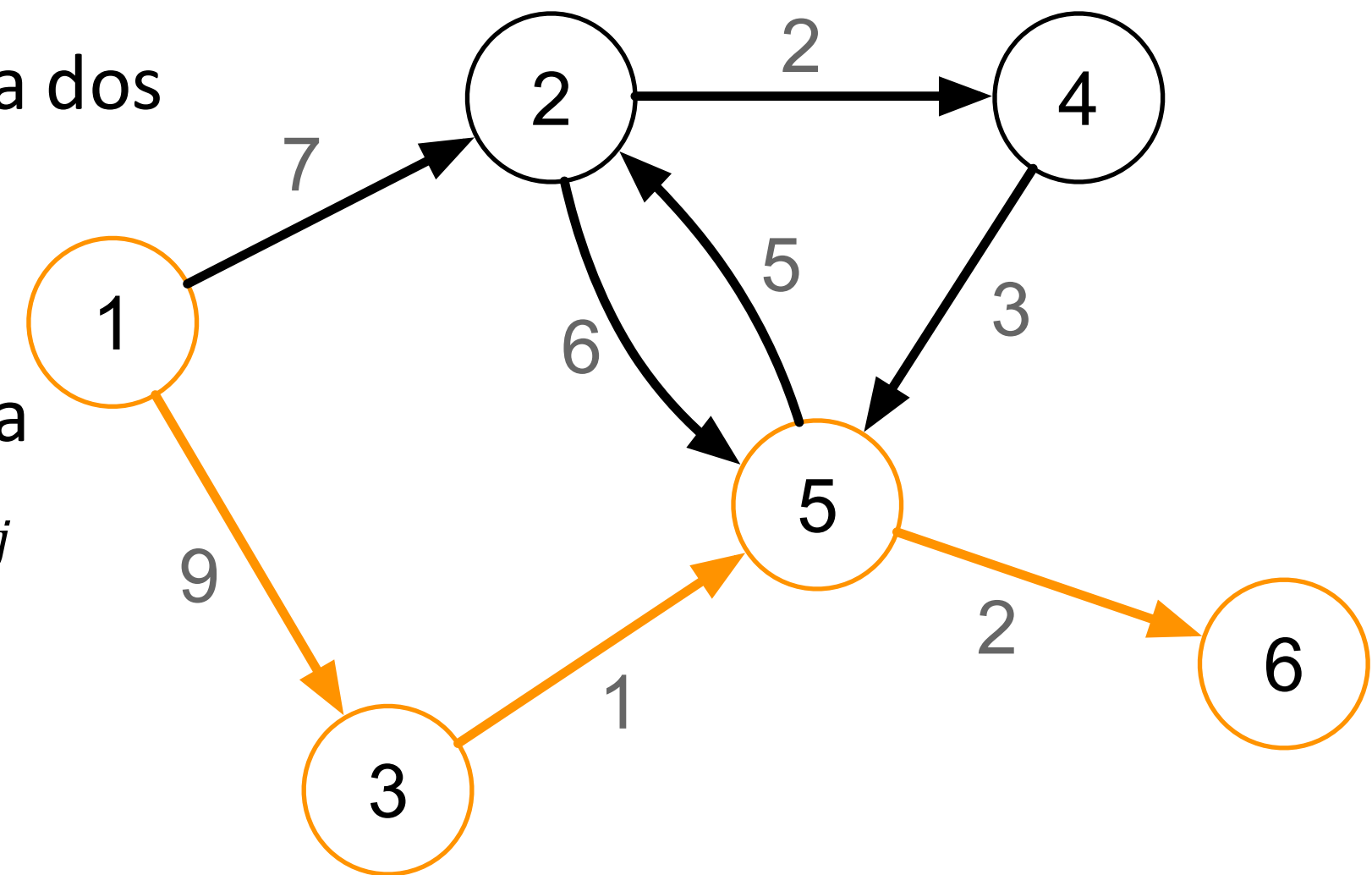
- Um grafo (orientado ou não) é ponderado se cada aresta estiver associada à um valor.
- O nome dessa propriedade varia de acordo com a natureza do problema
 - Custo.
 - Peso.
 - Capacidade.
 - Etc.



Menor caminho em Grafos

Caminho mais barato

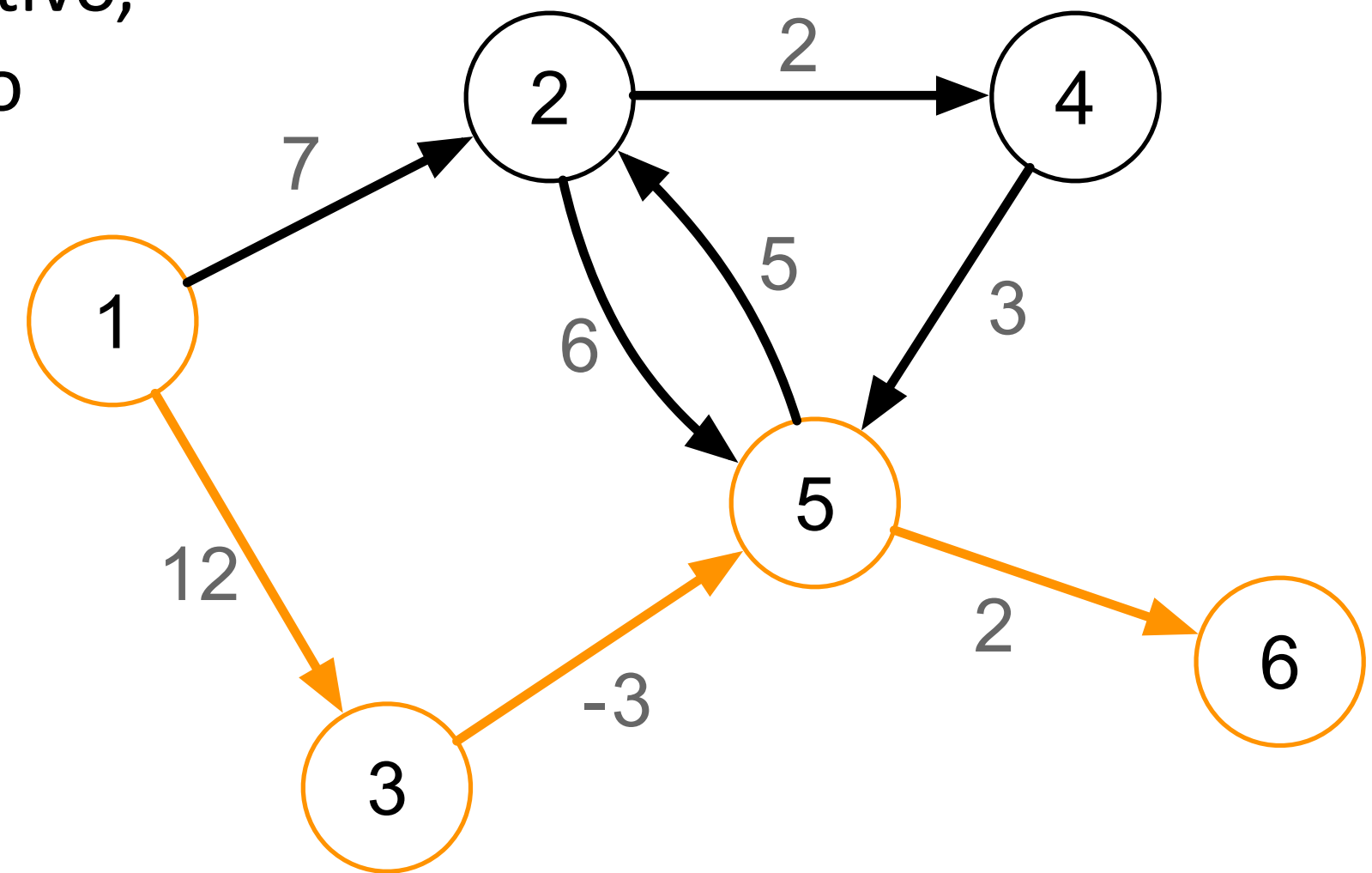
- **Problema:** dados dois vértices v_i e v_j em um grafo $G = (V, E)$ encontre o caminho P com o custo mínimo que começa em v_i e termina em v_j .
- O custo de um caminho é definido pela soma dos custos das arestas.
- O caminho tem custo mínimo caso não exista outro caminho em G entre os vértices v_i e v_j cujo custo seja menor que o de P .



Menor caminho em Grafos

Caminho mais barato

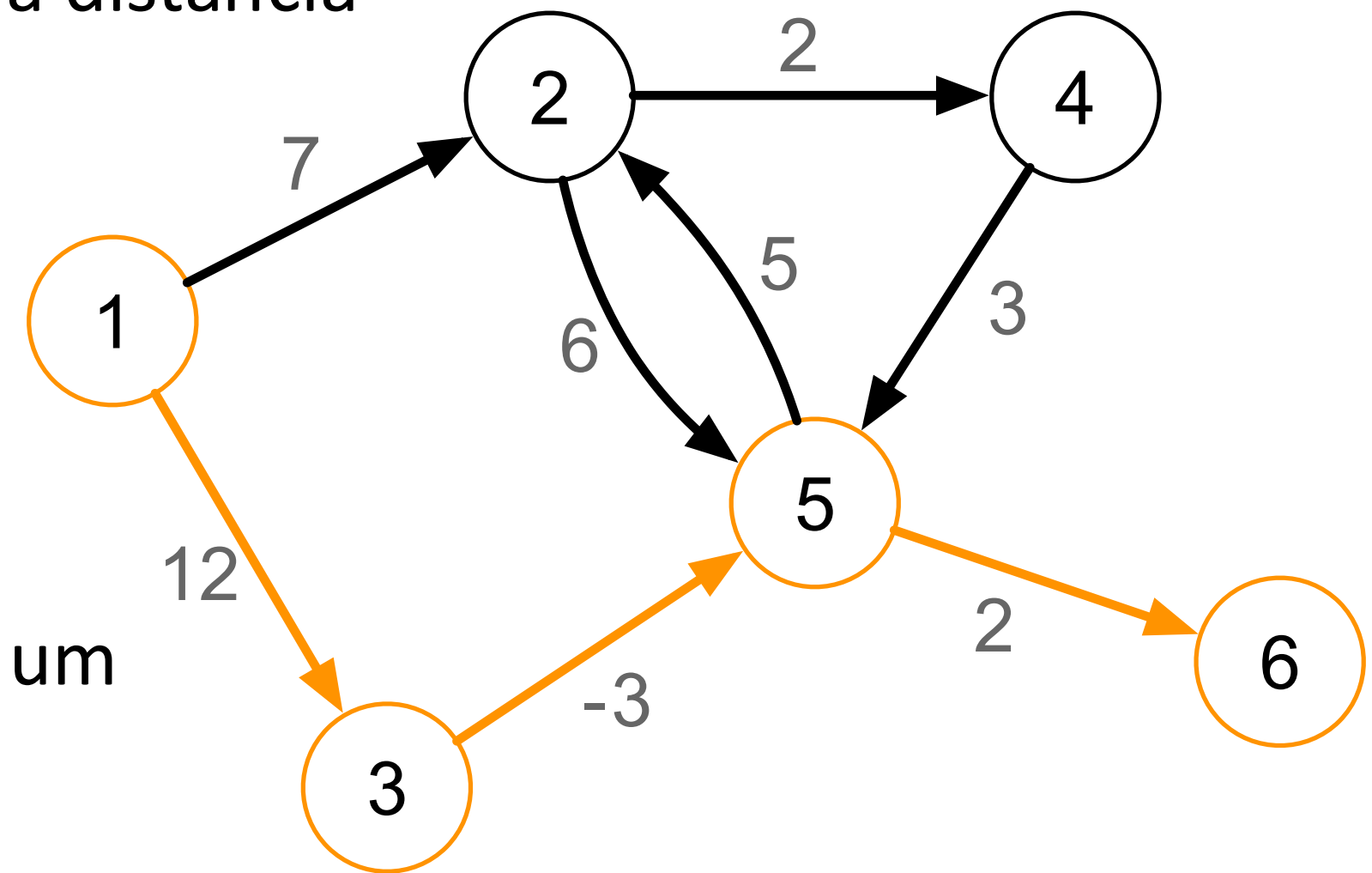
- Arestas podem apresentar **custos positivos e negativos**.
- Se todas as arestas apresentarem custo positivo, todo trecho de um caminho será mais barato que o caminho todo.
- Se existirem arestas com custo negativo, um trecho do caminho pode ser mais caro que todo o caminho.



Menor caminho em Grafos

Caminho mais barato

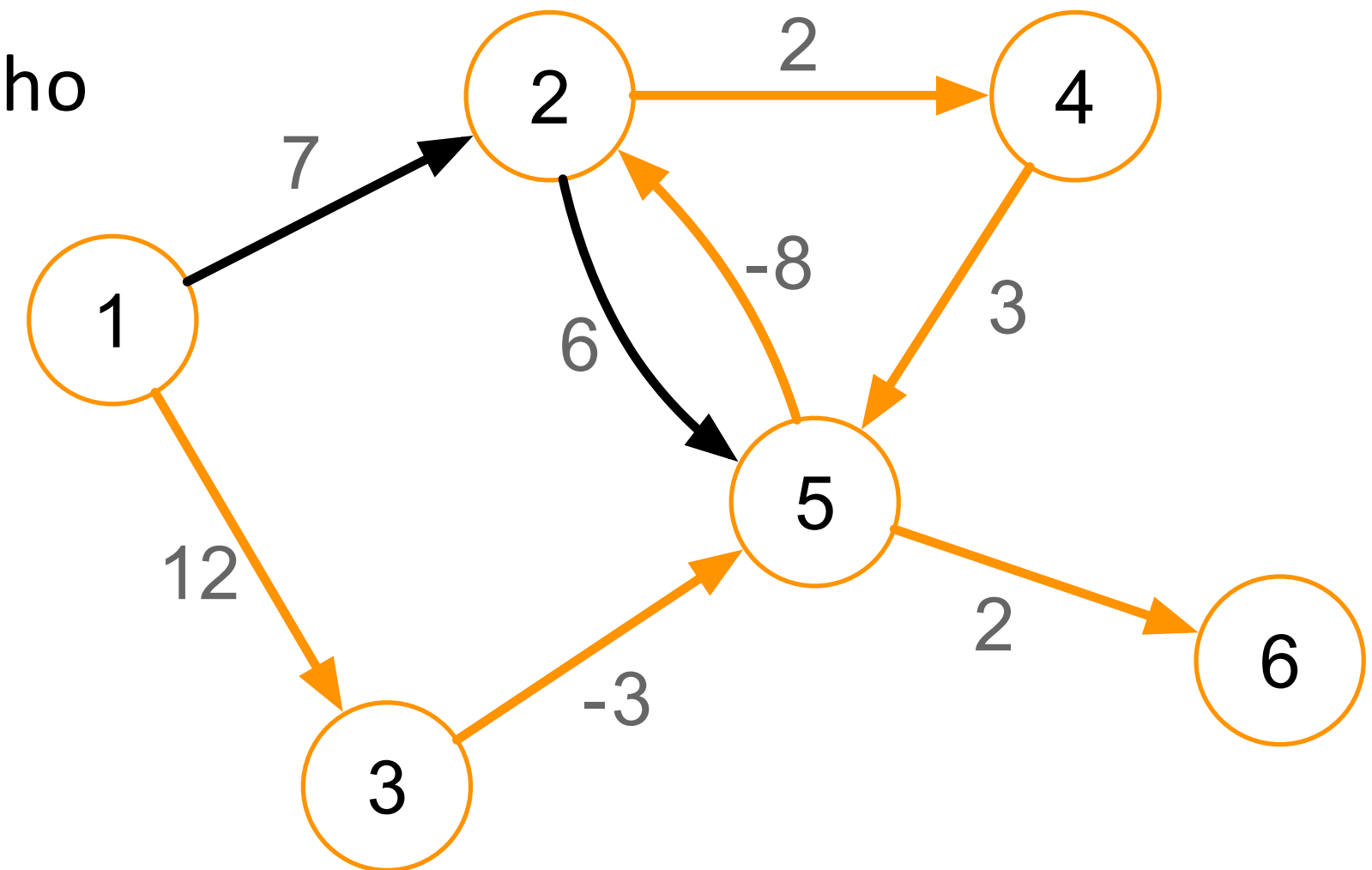
- A distância entre v_i e v_j é definida pelo comprimento do caminho mais barato.
- A distância entre v_i e v_j pode ser diferente da distância entre v_j e v_i caso o grafo seja orientado.
- Se não existir caminho entre v_i e v_j dizemos que a distância é infinita.
- Se existirem arestas negativas a distância de um caminho poderá ser negativa.



Menor caminho em Grafos

Caminho mais barato

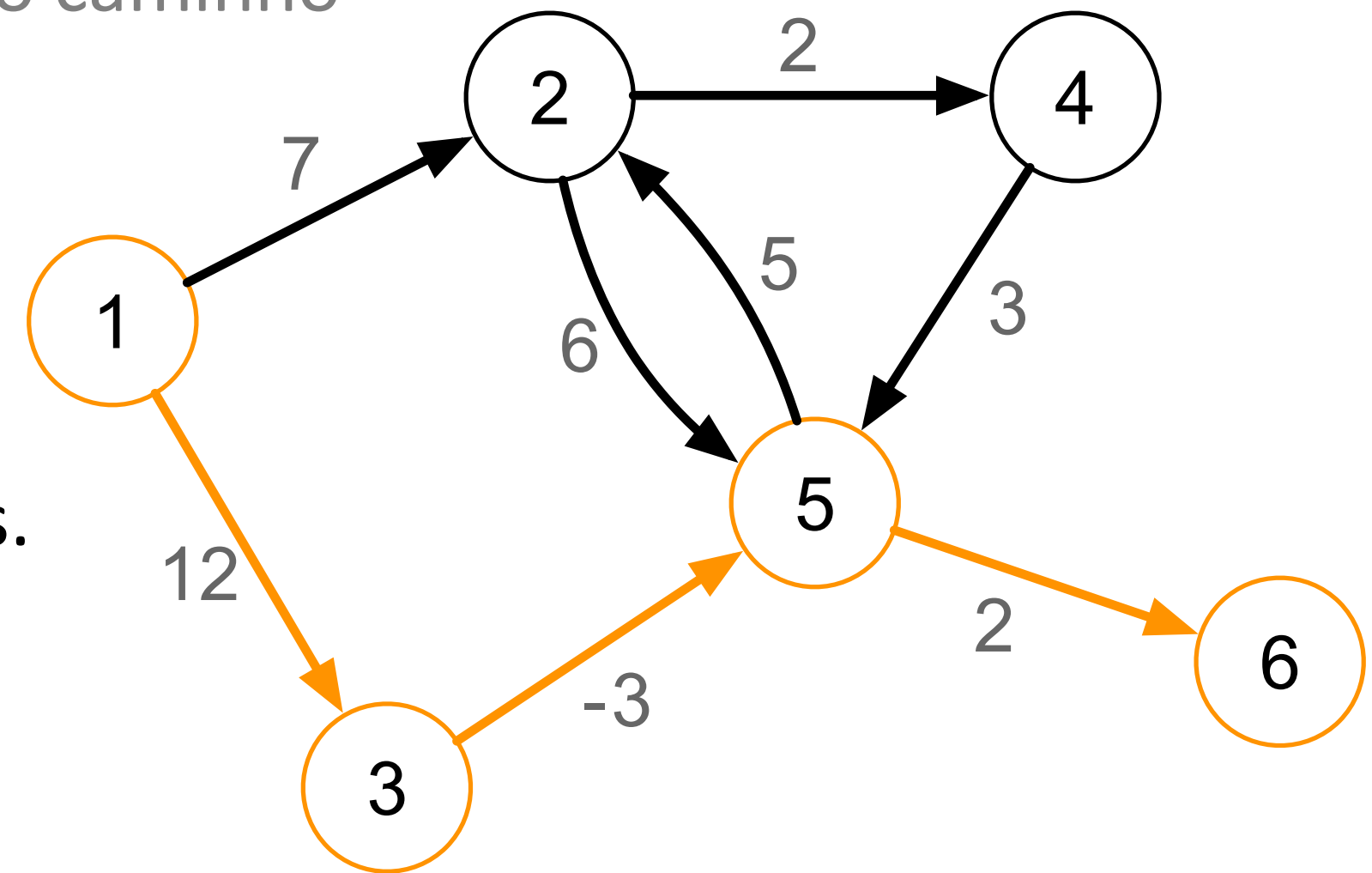
- Um **ciclo negativo** é um ciclo cujo custo resultante da soma das suas arestas é negativo.
- Se um grafo possuir ciclos negativos o caminho mais barato entre dois vértices pode não ser simples.
- Qual seria o custo do caminho em laranja na figura ao lado?



Menor caminho em Grafos

Caminho mais barato

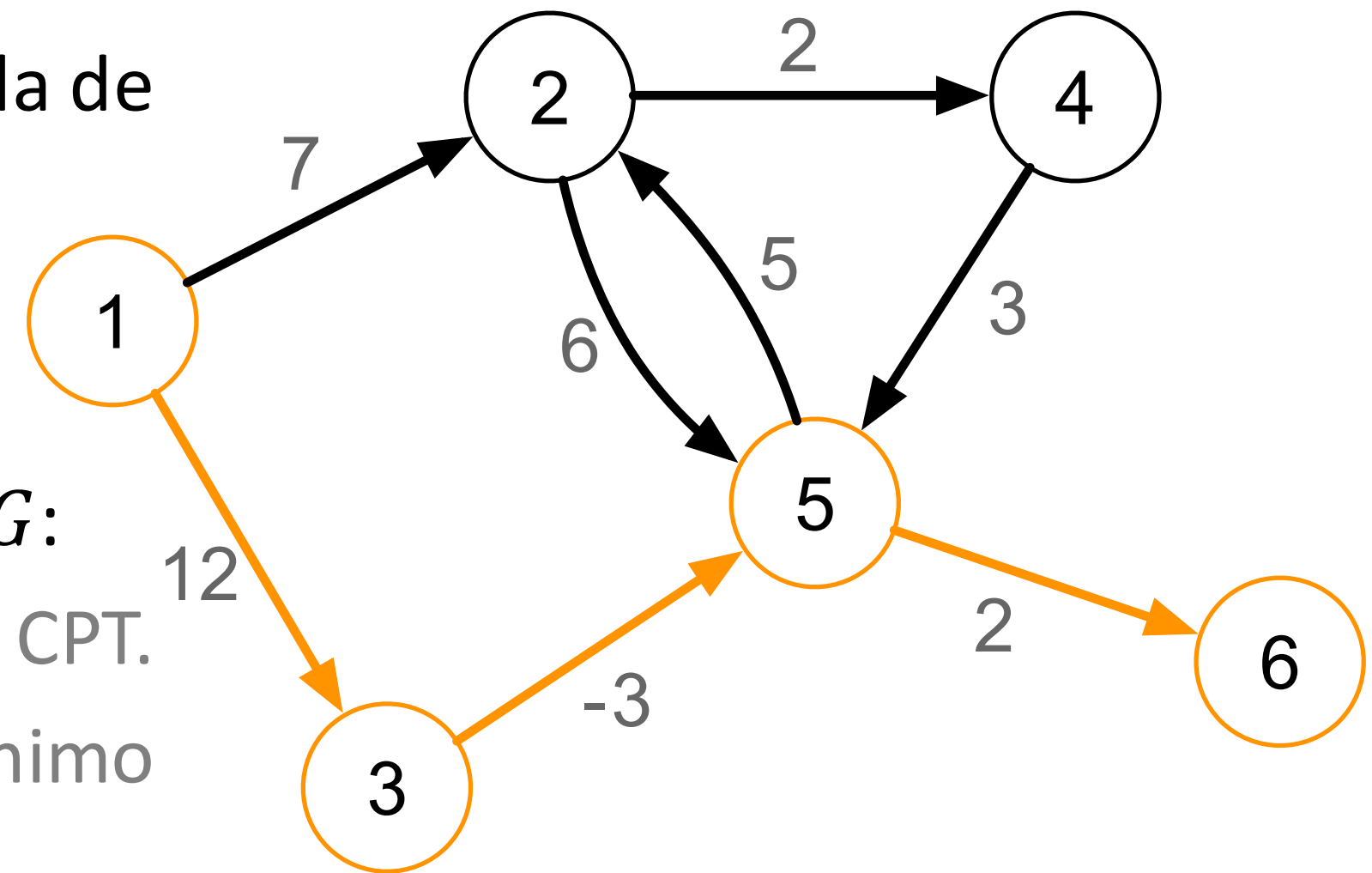
- Se v_0 é um vértice que não possui ciclos negativos ao seu alcance
 1. Todo caminho mais barato P com origem em v_0 é simples.
 2. Todo trecho inicial de P entre v_0 e v_k é o caminho mais barato de v_0 para v_k .
- Portanto, o problema de busca pelo caminho mais barato se torna muito mais simples quando não existem ciclos negativos.



Menor caminho em Grafos

Caminho mais barato

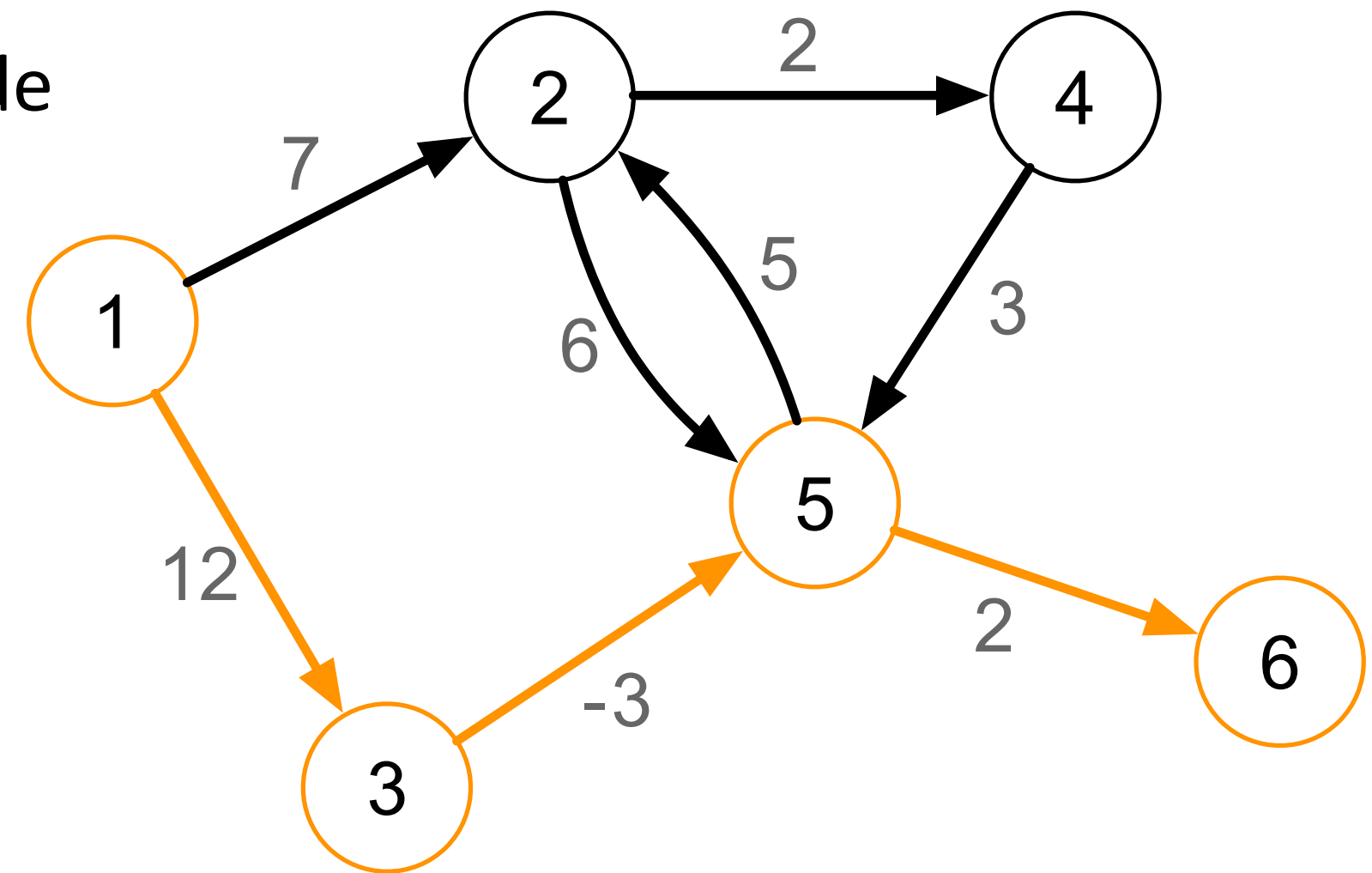
- Para encontrar o **caminho mais barato** entre v_i e v_j é inevitável encontrar todos os caminhos que iniciam em v_i .
- A árvore produzida por essa busca é chamada de Árvore de caminhos mais baratos (CPT – *Cheapest Path Tree*).
- A árvore CPT é uma sub-árvore radicada de G :
 - Todos os vértices de G estão presentes na CPT.
 - Todo caminho na CPT a partir da raiz é mínimo no grafo G .



Menor caminho em Grafos

Caminho mais barato

- Seja H o grafo induzido a partir de v_k , H somente possuirá uma CPT caso não possua ciclos negativos.
- Em grafos sem ciclos negativos o problema de encontrar o caminho de menor custo entre v_i e v_j se resume à encontrar a CPT com raiz em v_i .

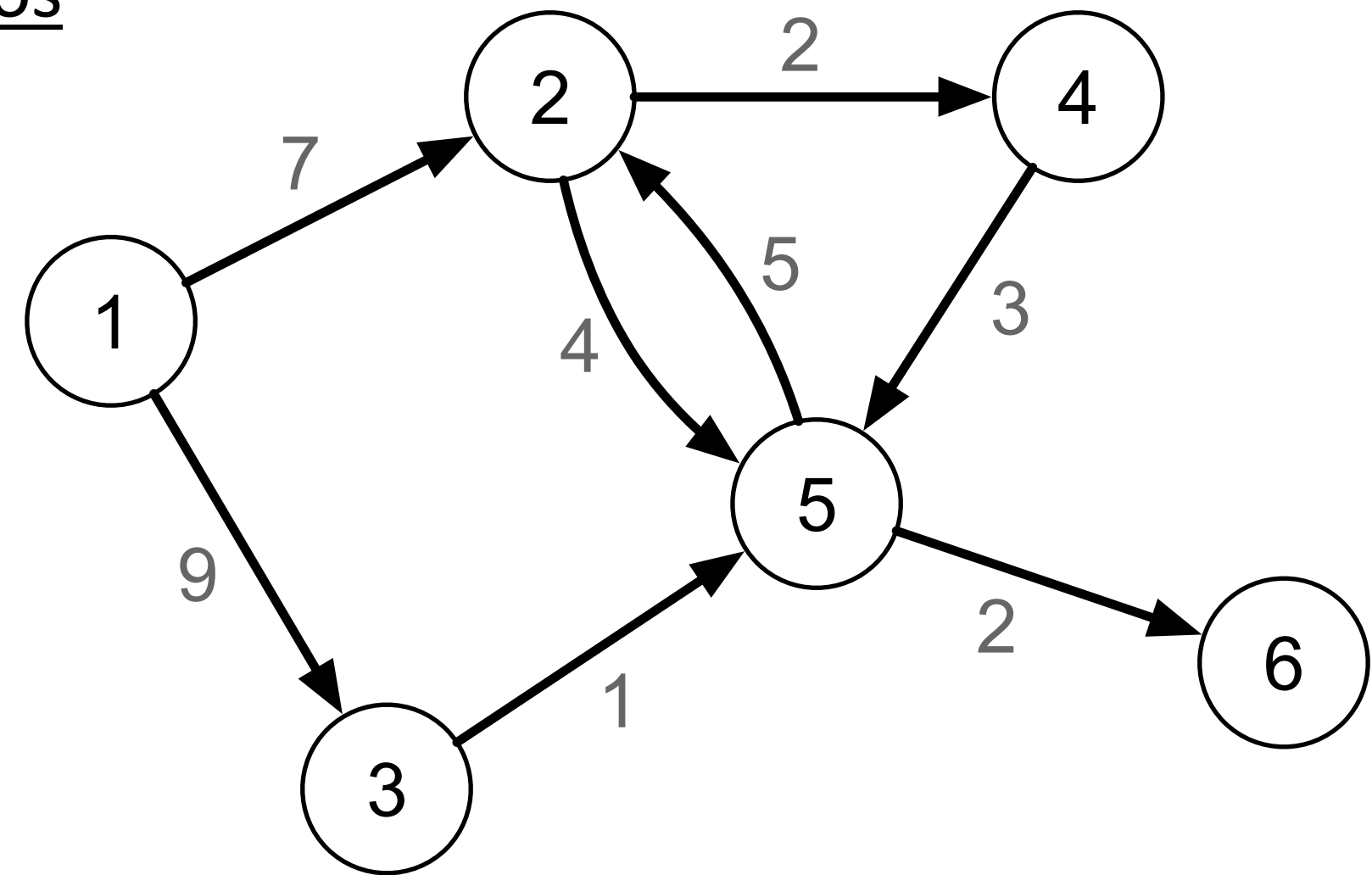


Dijkstra

Menor caminho em Grafos

Dijkstra

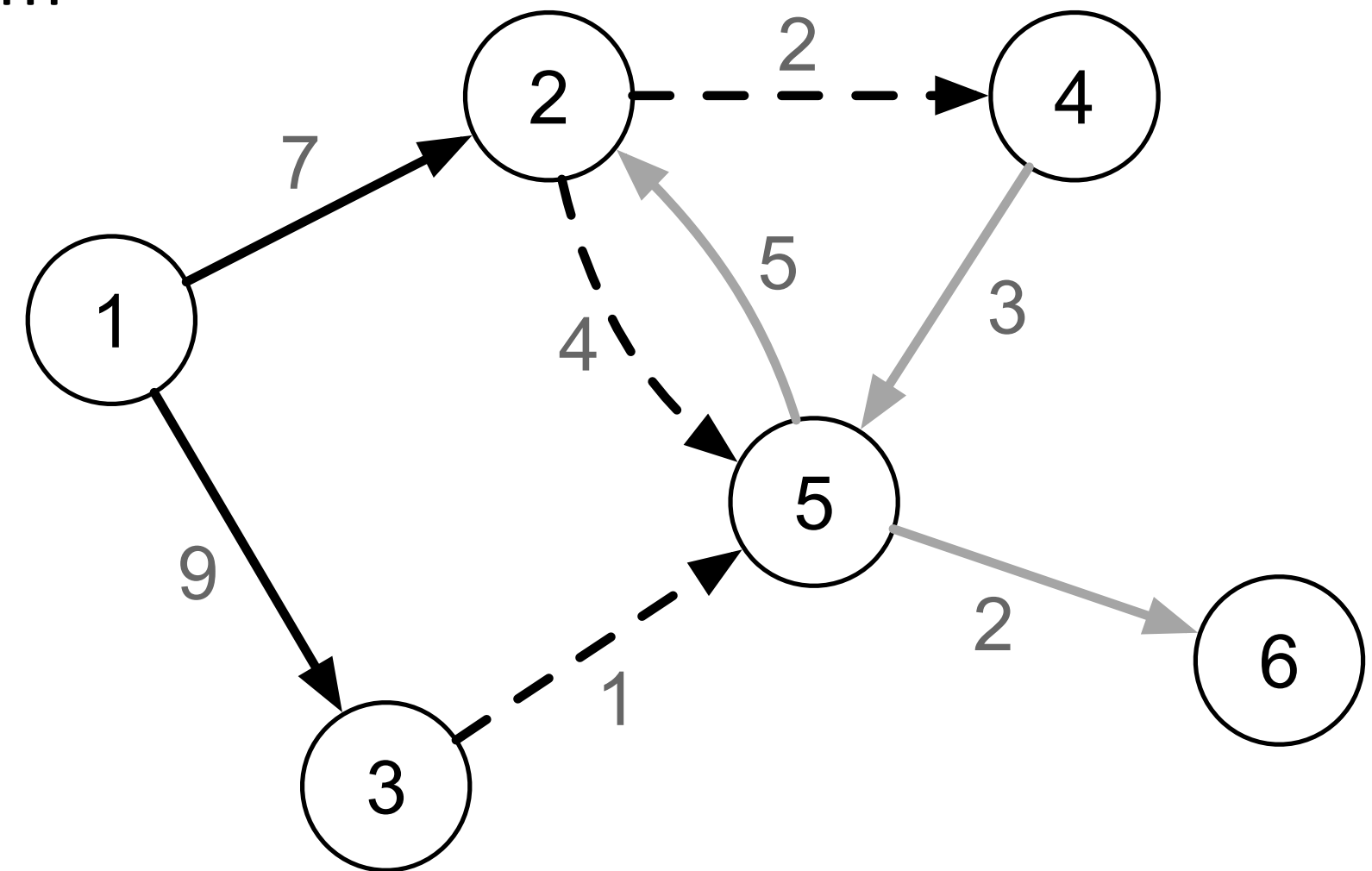
- O algoritmo de **Dijkstra** foi criado por Edsger W. Dijkstra em 1959.
- Esse algoritmo é capaz de encontrar caminhos mais baratos em um grafo $G = (V, E)$ que possua arestas com custos positivos.
- A solução consiste em crescer uma árvore radicada a partir do vértice inicial v_0 até que ela seja uma árvore geradora do subgrafo induzido a partir de v_0 .



Menor caminho em Grafos

Dijkstra

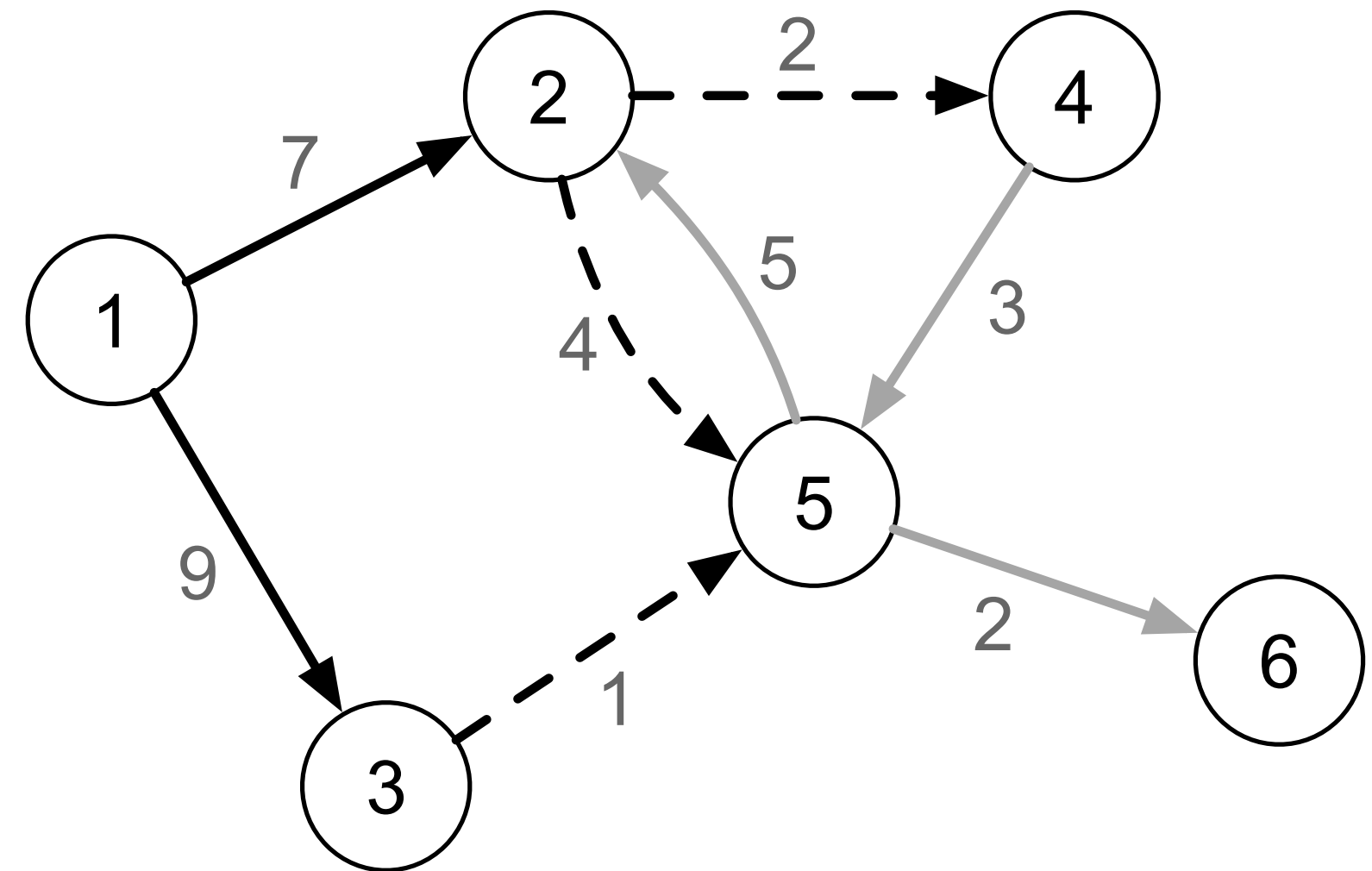
- Esse algoritmo introduz o conceito de **franja**.
- A franja de uma árvore radicada T com raiz em v_0 é o conjunto das arestas (v_i, v_j) que possuem v_i em T e v_j fora de T .
- A franja pode ser vista como o grau de saída do conjunto de vértices de T .



Menor caminho em Grafos

Dijkstra

- Ideia geral:
 - Insira o vértice v_0 em T e defina $d[v_0] = 0$.
 - Enquanto a franja não estiver vazia:
 - Escolha o vértice v_k que apresente a menor distância.
 - Aplique a operação de relaxamento em todas as arestas de v_k que pertencerem à franja.
 - Insira o vértice v_k em T .



Menor caminho em Grafos



Dijkstra

- Uma possível implementação seria:

```
void cptInit(vertex *parent, int *distance,
             bool *checked) const {
    for (vertex v=0; v < m_numVertices; v++) {
        parent[v] = -1;
        distance[v] = INT_MAX;
        checked[v] = false;
    }
}

void cptSeekLowest(const int *distance, const bool *checked,
                  int &minDistance, vertex &v1) const {
    for (vertex i=0; i < m_numVertices; i++) {
        if (checked[i]) continue;
        if (distance[i] < minDistance) {
            minDistance = distance[i];
            v1 = i;
        }
    }
}
```

```
void cptDijkstraSlow(vertex v0, vertex * parent,
                     int * distance) {
    bool checked[m_numVertices];
    cptInit(parent, distance, checked);
    parent[v0] = v0;
    distance[v0] = 0;
    while (true) {
        int minDistance = INT_MAX;
        vertex v1 = -1;
        cptSeekLowest(distance, checked, minDistance, v1);
        if (minDistance == INT_MAX) break;
        EdgeNode * edge = m_edges[v1];
        while (edge) {
            vertex v2 = edge->otherVertex();
            if (!checked[v2]) {
                int cost = edge->cost();
                if (distance[v1] + cost < distance[v2]) {
                    parent[v2] = v1;
                    distance[v2] = distance[v1] + cost;
                }
            }
            edge = edge->next();
        }
        checked[v1] = true;
    }
}
```

Dijkstra

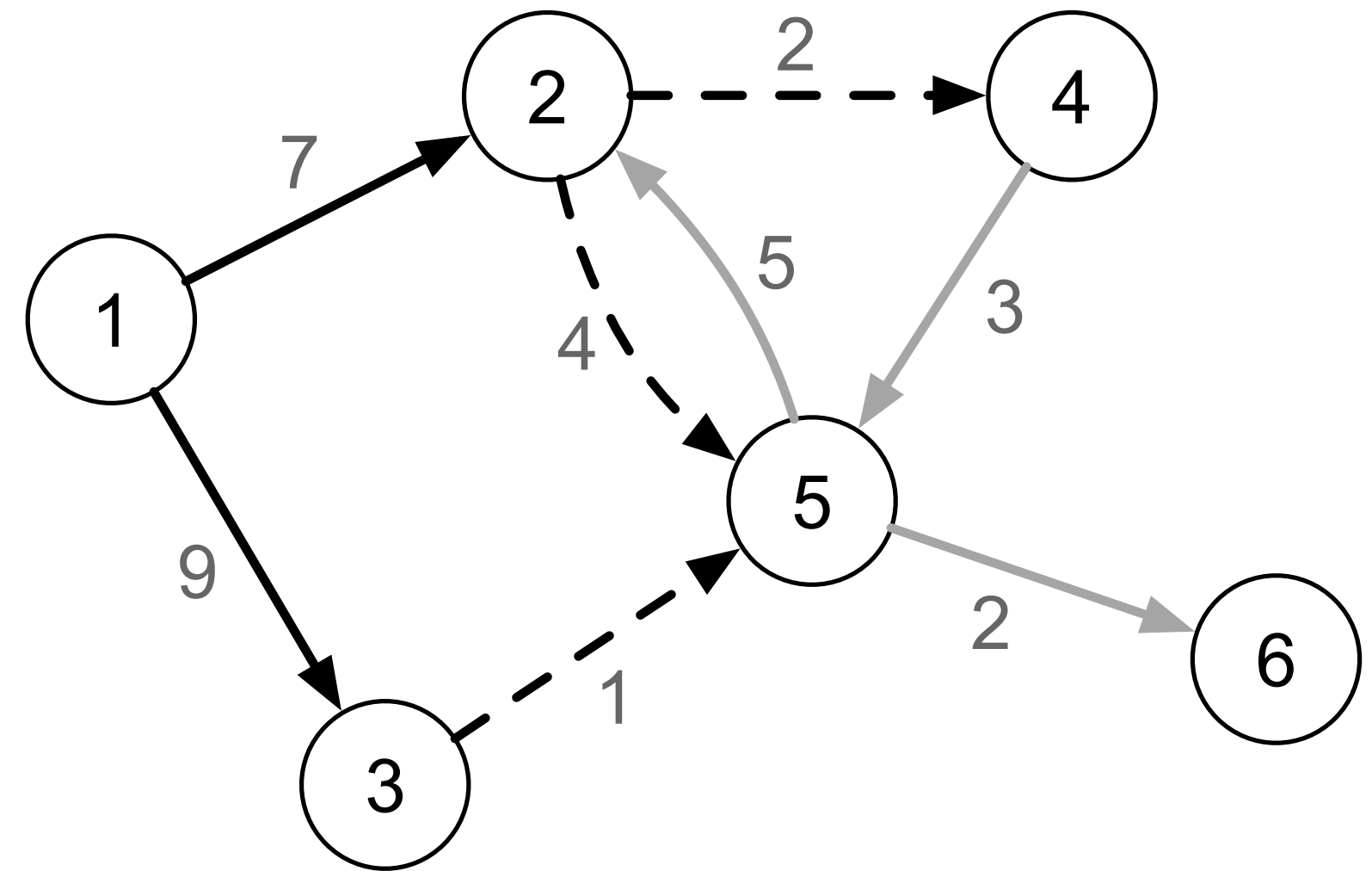
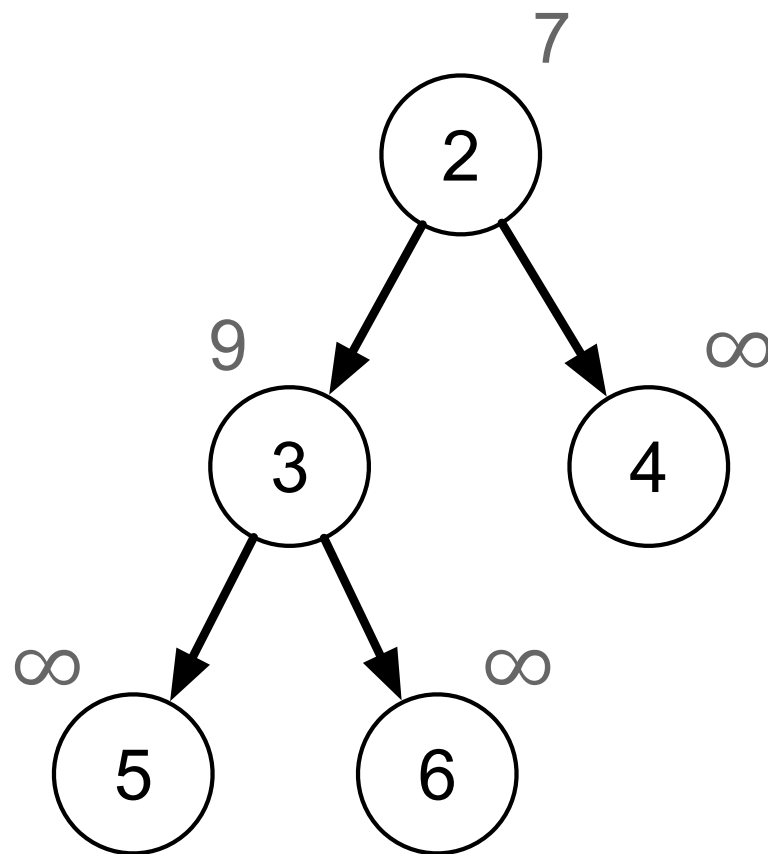
- Analise da complexidade:
 - Cada vértice é avaliado uma única vez.
 - Em cada avaliação:
 - Procura-se o vértice v_k em V de forma que $d[v_k]$ apresente a menor distância para v_0 .
 - Executa a operação de relaxamento em $g_s(v_k)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $O(V^2 + E)$.
- Sendo $E < V^2$, temos que a complexidade é $O(V^2)$.

- A implementação apresentada nos slides anteriores não é a mais eficiente.
- A cada iteração é buscado o vértice pertencente à franja que apresenta a menor distância para a raiz.
- Observe que a franja de uma iteração é similar à da iteração anterior.
- Como poderíamos melhorar o desempenho desse algoritmo com base nessa característica?

Menor caminho em Grafos

Dijkstra

- **Ideia:** manter os vértices da franja em uma fila de prioridades
 - Implementada como um heap mínimo.
 - Pode conter todos os vértices que ainda não foram verificados.



- **Exercício:** implemente a versão desse algoritmo baseada em fila de prioridades.

```
void cptDijkstraFast(vertex v0, vertex * parent,
                    int * distance) {
    bool checked[m_numVertices];
    Heap heap; // Create the heap
    for (vertex v=0; v < m_numVertices; v++) {
        parent[v] = -1;
        distance[v] = INT_MAX;
        checked[v] = false;
    }
    parent[v0] = v0;
    distance[v0] = 0;
```

```
    heap.insert_or_update(distance[v0], v0);
    while (!heap.empty()) {
        vertex v1 = heap.top().second; // Min vertex
        heap.pop(); // Remove from heap
        if (distance[v1] == INT_MAX) { break; }
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (!checked[v2]) {
                int cost = edge->cost();
                if (distance[v1] + cost < distance[v2]) {
                    parent[v2] = v1;
                    distance[v2] = distance[v1] + cost;
                    heap.insert_or_update(distance[v2], v2);
                }
            }
            edge = edge->next();
        }
        checked[v1] = true;
    }
}
```

* Essa implementação não insere todos os vértices na fila inicialmente.

- Analise da complexidade:
 - Cada vértice é avaliado uma única vez:
 - Obtêm-se o vértice v_i com menor distância na *heap* em $O(\log(V))$.
 - Executa-se a operação de relaxamento em $g_s(v_i)$ arestas, potencialmente adicionando v_j no *heap* em $O(\log(V))$.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $O((V + E) \log(V))$
 - Se todos os vértices forem acessíveis a partir de v_0 , temos $O(E \log(V))$.
 - Se o grafo for esparso, a complexidade será $O(V \log(V))$
 - Se o grafo for denso, temos que a complexidade é $O(V^2 \log(V))$
 - Ou seja, apresenta um desempenho inferior à abordagem anterior.

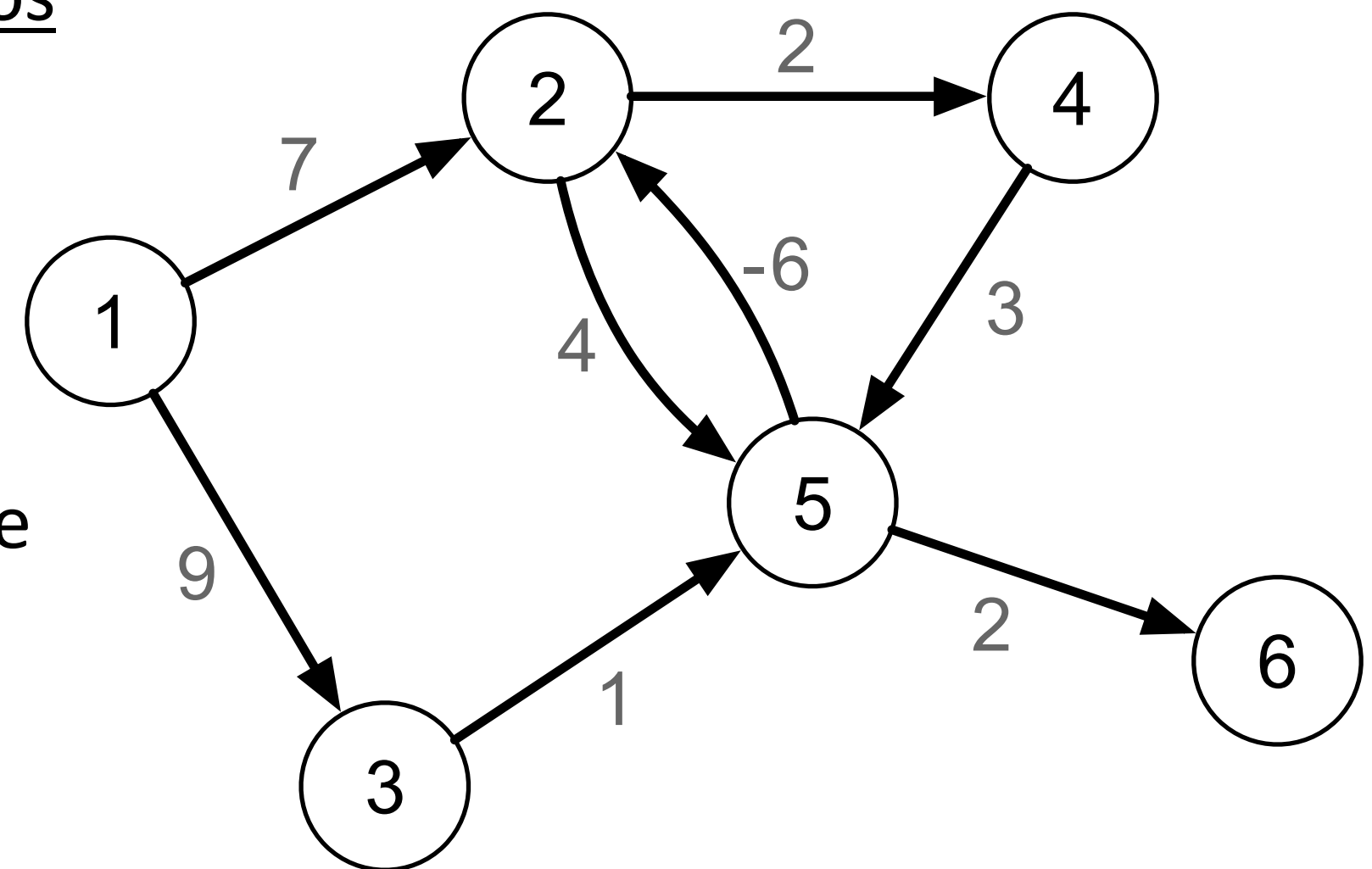
- Podemos avaliar a corretude desse algoritmo através das suas invariantes:
 1. A árvore T (definida por $parent$) representa uma árvore radcada com raiz em v_0 .
 2. Todas as arestas de T estão relaxadas e $distance[v] < \infty$ se e somente se $v \in T$.
 3. Toda aresta (v_i, v_j) de um vértice verificado v_i está relaxada em relação à $distance$.
- Ao término da execução:
 - Todos os vértices foram verificados.
 - A partir da invariante (3) todas as arestas com v_i em T estão relaxadas
 - A partir do invariante (2) sabemos que $distance[v_j] < \infty$, portanto $v_j \in T$.
 - Todo vértice acessível a partir de v_0 está em T , logo T é geradora do subgrafo induzido H .
 - Dado que $distance$ representa a menor distância de v_0 à qualquer outro vértice de T , temos que T é a CPT de H com raiz em v_0 .

Bellman-Ford

Menor caminho em Grafos

Bellman-Ford

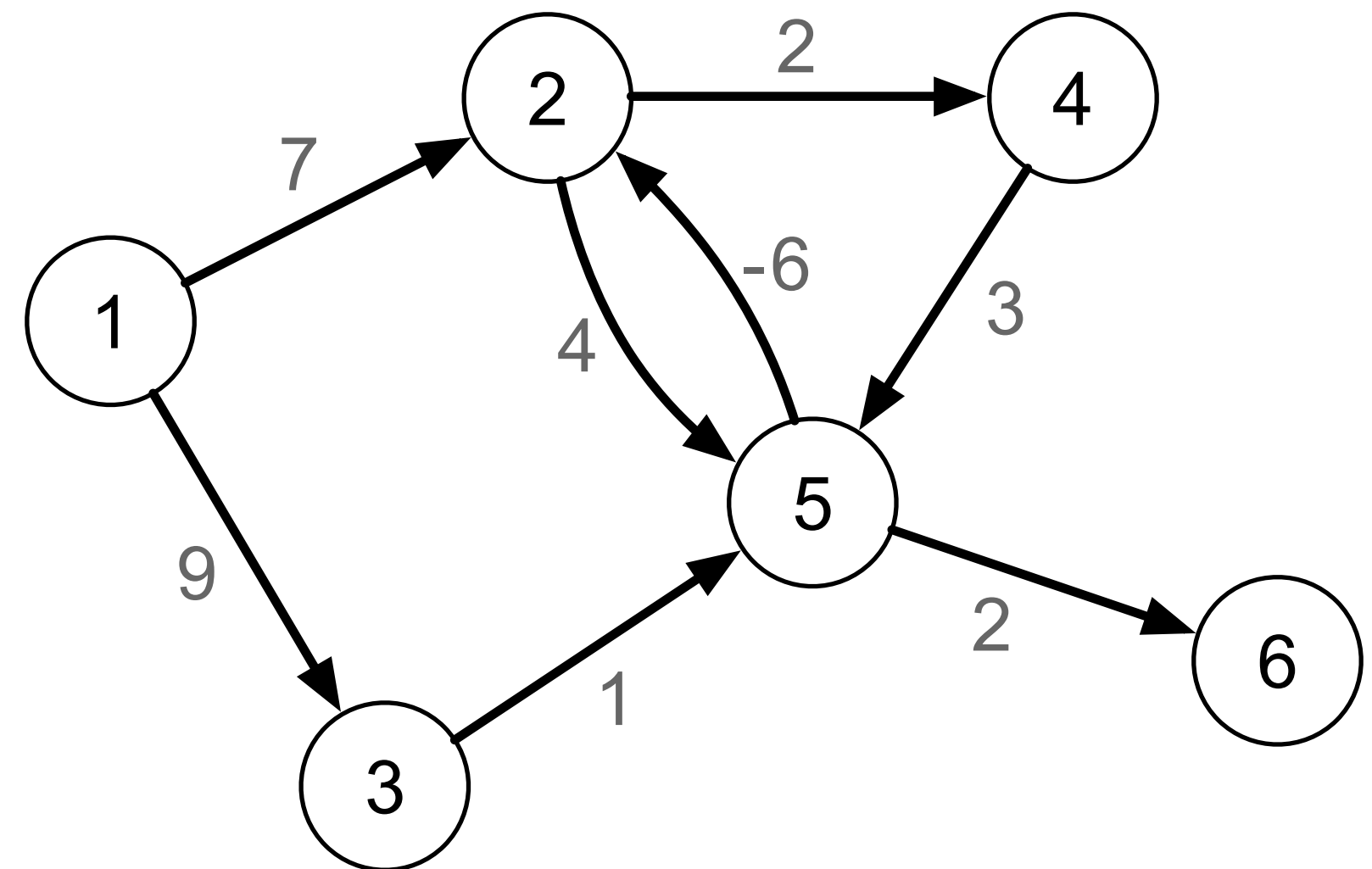
- O algoritmo de **Bellman-Ford** foi criado por Richard Bellman e Lester Ford em 1956.
- Esse algoritmo é capaz de encontrar caminhos mais baratos em um grafo $G = (V, E)$ que possua arestas com custos positivos e negativos.
- Ao término da execução o algoritmo indica se foi encontrada a CPT ou foi detectado um ciclo negativo.



Menor caminho em Grafos

Bellman-Ford

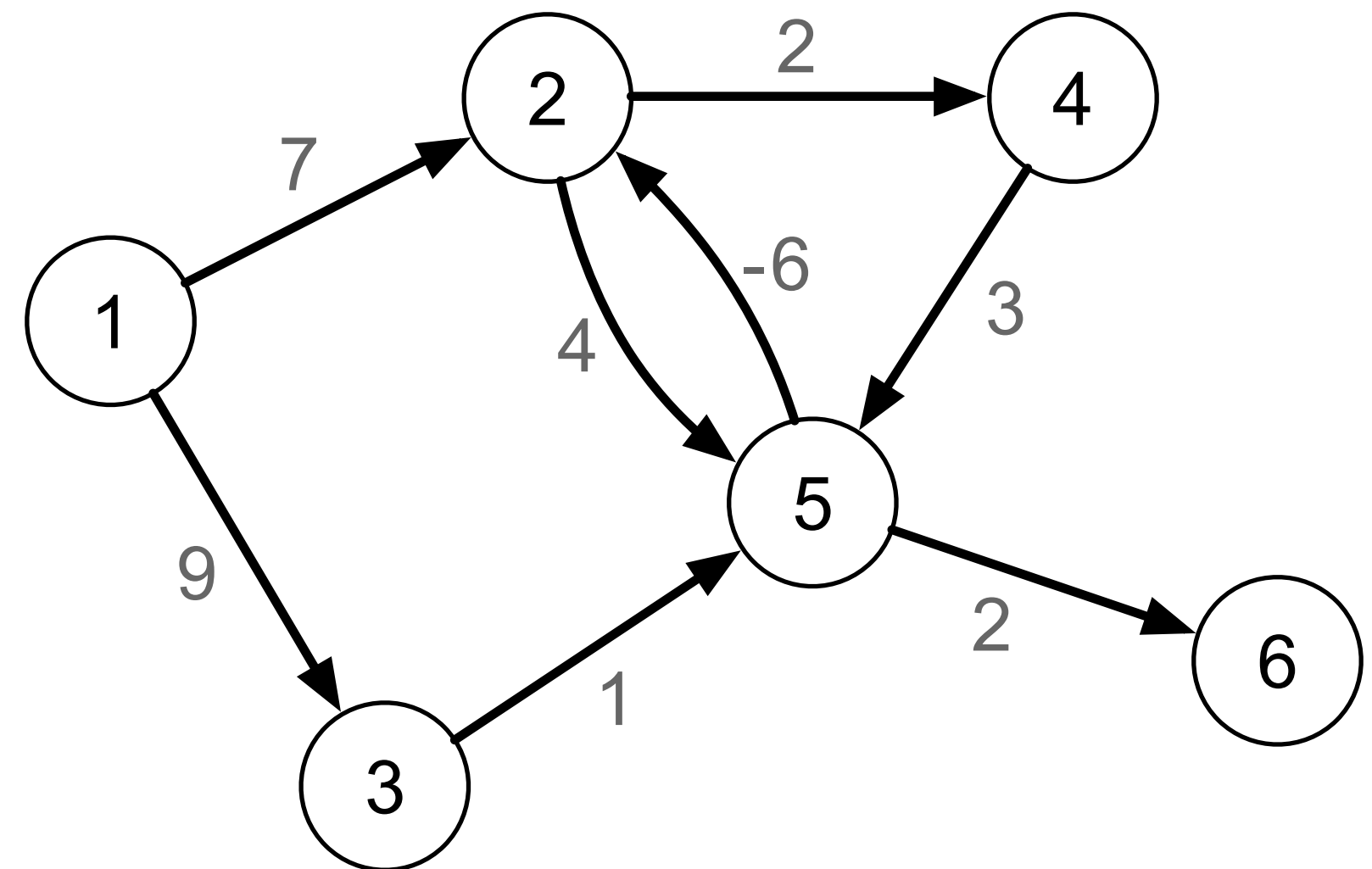
- A solução consiste em relaxar as arestas do grafo sistematicamente, reduzindo progressivamente uma estimativa $d[v]$ para cada vértice $v \in V$ do grafo, até alcançar a menor distância.
- O retorno do algoritmo consiste em:
 - Um indicador de sucesso
 - Ausência de ciclos negativos.
 - Uma árvore representando a CPT.
 - A distância para cada vértice.



Menor caminho em Grafos

Bellman-Ford

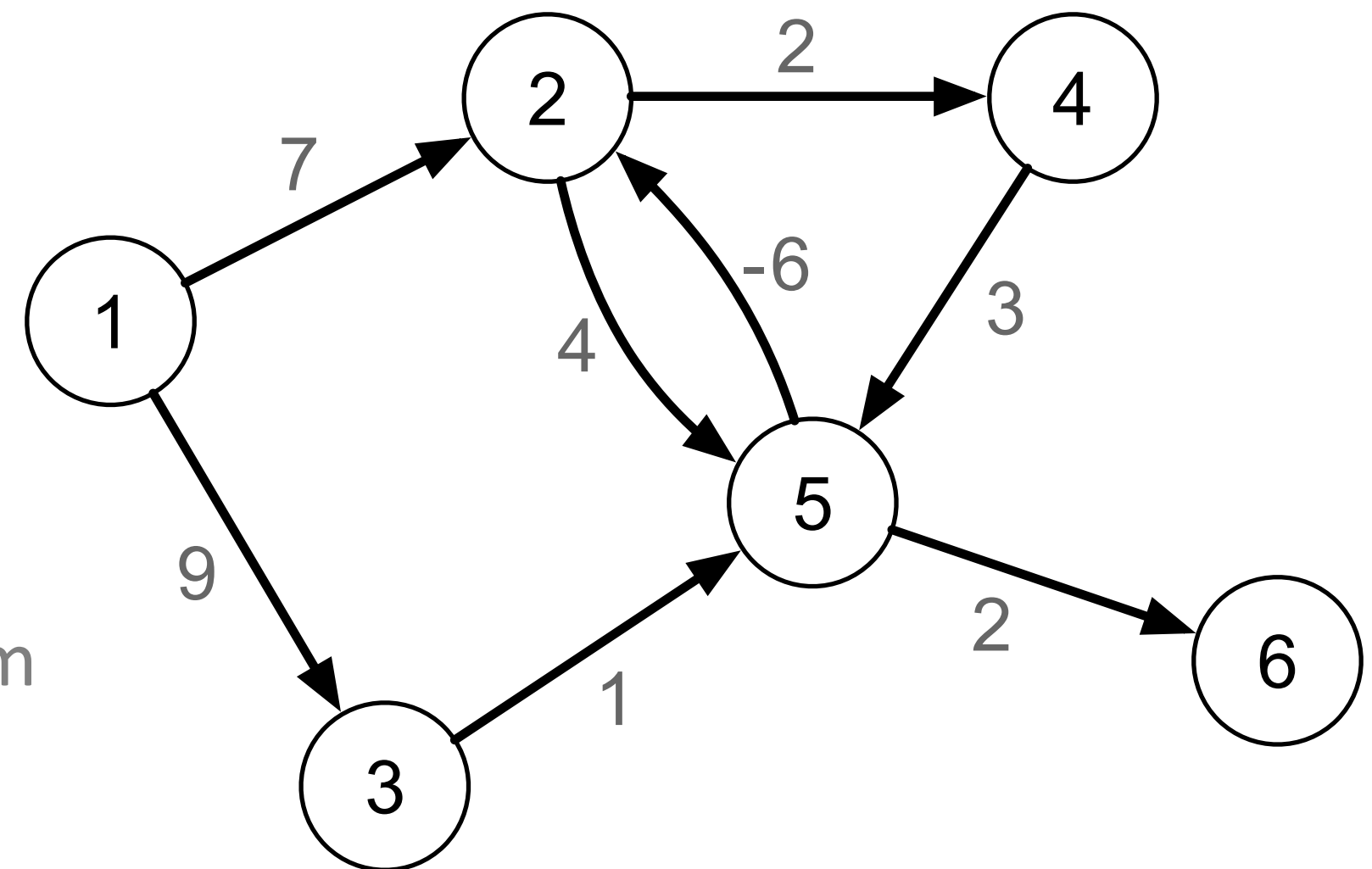
- Ideia geral:
 - Insira o vértice v_0 em T e defina $d[v_0] = 0$.
 - Execute $V - 1$ vezes:
 - Para cada aresta (v_i, v_j)
 - Aplique o relaxamento.
 - Execute uma nova iteração de relaxamento sobre todas as arestas
 - Se alguma distância $d[v_k]$ for reduzida, um ciclo negativo foi detectado.



Menor caminho em Grafos

Bellman-Ford

- O algoritmo executa as primeiras $V - 1$ iterações construindo caminhos com:
 - 1 aresta
 - 2 arestas
 - Até $V - 1$ arestas.
- Considerando que um caminho simples pode ter no máximo $V - 1$ arestas:
 - Se a iteração V encontrar uma distância menor para algum vértice, significa que um ciclo negativo foi encontrado.
 - Caso contrário o grafo não possui ciclos negativos e temos a CPT.

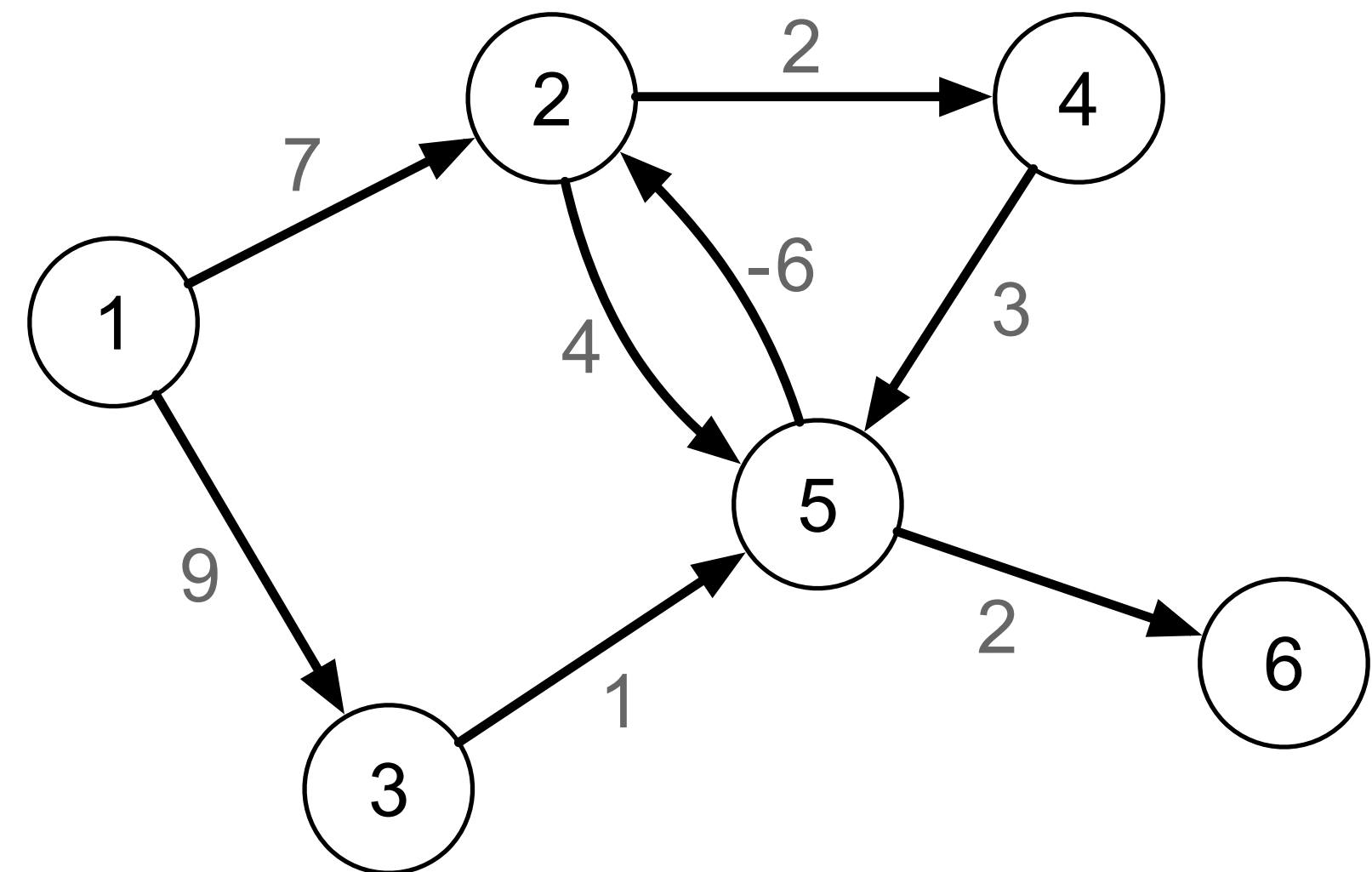


Menor caminho em Grafos

Bellman-Ford

- Como seria o estado do vetor de distâncias ao aplicar o algoritmo sobre o grafo a seguir?

0	0	–	–	–	–	–
1	0	4	9	9	10	12
2	0	2	9	6	8	10
3	0	0	9	4	6	8
4	0	–2	9	2	4	6
5	0	–4	9	0	2	4
6	0	–4	9	0	0	4



- **Exercício:** implemente o algoritmo de Bellman-Ford.

```
bool cptBellmanFord(vertex v0, vertex * parent,
                    int * distance) {
    for (int v=0; v < m_numVertices; v++) {
        parent[v] = -1;
        distance[v] = INT_MAX;
    }
    parent[v0] = v0;
    distance[v0] = 0;
    for (int i=1; i <= m_numVertices - 1; i++) {
        for (int v1 = 0; v1 < m_numVertices; v1++) {
            EdgeNode * edge = m_edges[v1];
            while(edge) {
                vertex v2 = edge->otherVertex();
                int cost = edge->cost();
                if (distance[v1] + cost < distance[v2]) {
                    parent[v2] = v1;
                    distance[v2] = distance[v1] + cost;
                }
                edge = edge->next();
            }
        }
    }
}
```

```
for (int v1=0; v1 < m_numVertices; v1++) {
    EdgeNode * edge = m_edges[v1];
    while(edge) {
        vertex v2 = edge->otherVertex();
        int cost = edge->cost();
        if (distance[v1] + cost < distance[v2]) {
            return false;
        }
        edge = edge->next();
    }
}
return true;
```

A complexidade desse algoritmo é: $O(VE)$

Perguntas?

`thiago.p.araujo@fgv.br`